UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE GRADO

GRADO EN TECNOLOGÍAS PARA LA SOCIEDAD DE LA INFORMACIÓN

SOLARIS: Simulador de módulos fotovoltaicos con inyección y composición de fallos

Desarrollado por: Ionut Alexandru Ona Marisescu

Dirigido por: Dr. Iglesias Hernández, Guillermo

Madrid, 12 de julio de 2026

SOLARIS: Simulador de módulos fotovoltaicos con inyección y composición de fallos
Desarrollado por: Ionut Alexandru Ona Marisescu
Dirigido por: Dr. Iglesias Hernández, Guillermo
Proyecto Fin de Grado, 12 de julio de 2026

E.T.S. de Ingeniería de Sistemas Informáticos
Campus Sur UPM, Carretera de Valencia (A-3), km. 7
28031, Madrid, España

Si deseas citar este trabajo, la entrada completa en BibTeX es la siguiente:

```
@mastersthesis{citekey,
title = {SOLARIS: Simulador de módulos fotovoltaicos con inyección y composición
de fallos},
author = {Ona Marisescu, I. A. y Dr. Iglesias Hernández, G.},
school = {E.T.S. de Ingeniería de Sistemas Informáticos},
year = {2026},
month = {7},
type = {Proyecto Fin de Grado}
}
```

Esta obra está bajo una licencia Creative Commons «Atribución-NoComercial-CompartirIgual 4.0 Internacional». Obra derivada de https://github.com/blazaid/UPM-Report-Template.



Todo cambio respecto a la obra original es responsabilidad exclusiva del presente autor.

Agradecimientos

Como siempre, a mis padres, María Angélica Ona y Ioan Sorin Ona, por estar siempre conmigo y por mí.
A mi hermano, Alberto Gabriel Ona, por los buenos años pasados y los que vendrán.
A los cinco muchachos, que ya son familia.
A aquellos que se quedaron en el norte, que incluso siguiendo su propio camino me acompañan en el mío.
A los profesores que me han acompañado y ayudado a crecer en este recorrido.

A Ángel Palomeque Reyes, que fue siempre un más que digno rival, y aún mejor amigo. Sigues y seguirás presente por siempre.

Resumen

La detección de fallos en módulos fotovoltaicos es un factor clave para la viabilidad económica de la energía solar, y un campo de investigación activo. Sin embargo, el desarrollo y la validación de estos algoritmos se ven limitados por la escasez de datos públicos anotados. Los conjuntos experimentales existentes son costosos de producir, rígidos y ligados a instalaciones concretas, mientras que las herramientas de simulación consolidadas modelan el comportamiento sano con gran fidelidad pero tratan la degradación como una corrección agregada, sin capacidad de inyectar modos de fallo individuales con progresión temporal realista.

SOLARIS es un simulador extensible de módulos fotovoltaicos con inyección y composición de fallos, orientado a la generación de datos sintéticos anotados. El sistema se fundamenta en el modelo de diodo único, con parámetros de la base de datos CEC y datos meteorológicos de fuentes institucionales, y define una interfaz abstracta mediante la cual cada efecto, fallo o modo de degradación declara los parámetros físicos que perturba, delegando en un motor de composición la combinación de efectos múltiples. La ejecución se paraleliza mediante multiproceso con reparto de carga acotado por memoria, y la reproducibilidad se garantiza mediante derivación determinista de semillas por par panel-efecto. Se demuestra la viabilidad del sistema simulando 10.000 paneles sobre una serie temporal de 16 años en hardware de consumo, con un análisis de escalabilidad frente al número de procesos y del coste computacional de los efectos, y se valida la arquitectura implementando un modelo de acumulación de suciedad y demostrando la composición de dos instancias simultáneas de este sobre la producción de la flota.

El código se puede encontrar en: https://github.com/MewKitBit/SOLARIS

Palabras clave: energía fotovoltaica, simulación, detección de fallos, datos sintéticos, modelo de diodo único, degradación, aprendizaje automático, paralelización

Abstract

Early fault detection in photovoltaic modules is a key factor in the economic viability of solar energy, and an active research field. However, the development and validation of these algorithms is constrained by the scarcity of annotated public data. Existing experimental datasets are costly to produce, rigid, and tied to specific installations, while established simulation tools model healthy behavior with high fidelity but treat degradation as an aggregate correction, lacking the ability to inject individual failure modes with realistic temporal progression.

SOLARIS is an extensible photovoltaic module simulator with fault injection and composition, aimed at generating annotated synthetic data. The system is built on the single-diode model, with parameters from the CEC database and meteorological data from institutional sources, and defines an abstract interface through which each effect, fault, or degradation mode declares the physical parameters it perturbs, delegating the combination of multiple effects to a composition engine. Execution is parallelized via multiprocessing with memory-bounded load distribution, and reproducibility is guaranteed through deterministic seed derivation per panel-effect pair. System viability is demonstrated by simulating 10,000 panels over a 16-year time series on consumer hardware, with a scalability analysis against worker count and effect count, and the architecture is validated by implementing a soiling accumulation model and demonstrating the composition of two simultaneous instances on fleet-level output.

The code can be found at: https://github.com/MewKitBit/SOLARIS

Keywords: photovoltaics, simulation, fault detection, synthetic data, single-diode model, degradation, machine learning, parallelization

Índice general

1 Introducción **1**

1.1 Estructura de la memoria 1

1.2 Estado del arte 1

1.3 Motivación y Justificación 4

1.4 Objetivos de Desarrollo Sostenible 7

1.5 Objetivos 8

2 Conocimiento Teórico y Técnico **10**

2.1 Módulos fotovoltaicos 10

2.2 Paralelización 17

2.3 Herramientas y Librerías 20

3 Descripción del Sistema **26**

3.1 Arquitectura del proyecto 26

3.2 Diagrama de flujo de datos 28

3.3 Configuración y Ejecución 29

3.4 Simulación 38

3.5 Formato de Resultados 48

4 Demostración **50**

4.1 Testing 50
4.2 Implementación de un efecto 51
4.3 Análisis de rendimiento 53

5 Conclusiones **58**

5.1 Análisis frente a los objetivos establecidos 58
5.2 Conclusiones 61
5.3 Líneas futuras 61

Índice de figuras

2.1 Diagrama explosionado de un módulo fotovoltaico [15]. 10

2.2 Curva I-V de un módulo comercial común bajo condiciones estándar. 11

2.3 Diagrama de circuito equivalente SDM[16]. 13

2.4 Esquema diferenciativo entre múltiples hilos y múltiples procesos[24]. 19

3.1 Diagrama de flujo de datos de SOLARIS. 28

4.1 Producción energética por módulos fotovoltaicos sin acumulación de suciedad. 52

4.2 Producción energética por módulos fotovoltaicos con acumulación de suciedad. 52

4.3 Producción energética por módulos fotovoltaicos con acumulación de suciedad en dos fases. 53

Índice de tablas

3.1 Parámetros susceptibles a efectos y reglas de composición. 41

3.2 Columnas del conjunto de observables. 49

4.1 Tiempo de ejecución, rendimiento, aceleración y eficiencia de la simulación (10000 paneles, 16 años, un único efecto) en función del número de procesos de trabajo. 54

4.2 Tiempo de ejecución y rendimiento de la simulación en función del número de efectos activos. 56

Índice de listados

3.1 Configuración de ubicación y periodo en fichero `TOML` 30

3.2 Configuración de parámetros de ejecución (`runtime`) 31

3.3 Configuración de rutas 32

3.4 Configuración de modelos físicos 33

3.5 Configuración de módulo CEC y montaje físico 34

3.6 Configuración de efectos, fallos y modos de degradación 35

3.7 Comando de CLI para la adquisición de datos de entrada de SOLARIS 37

3.8 Comando de CLI para la ejecución de SOLARIS 37

1. Introducción

La siguiente sección sirve como introducción al proyecto SOLARIS, dando información general, la motivación y justificación detrás de este, los objetivos planteados y la estructura definida en esta memoria.

1.1. Estructura de la memoria

Este documento está compuesto de las siguientes partes:

- **Introducción:** Presenta el contexto, la motivación, justificación, estado del arte y los objetivos del proyecto.
- **Conocimiento Teórico y Técnico:** Introduce y explica brevemente el trasfondo teórico y técnico sobre el que se construye el proyecto.
- **Descripción del Sistema:** Detalla el sistema desarrollado. Funciona como suplemento a la documentación del proyecto.
- **Demostración y Validación:** Demuestra el funcionamiento del sistema y compara el resultado preliminar del proyecto con los objetivos establecidos.
- **Conclusiones:** Presenta las conclusiones y líneas futuras del proyecto.

1.2. Estado del arte

Las técnicas de detección de fallos en sistemas fotovoltaicos, los conjuntos de datos públicos disponibles para su desarrollo y validación, y las herramientas

de simulación fotovoltaica existentes revelan una carencia en la intersección de tres áreas analizadas en las siguientes subsecciones.

Los conjuntos de datos experimentales existentes son valiosos pero difíciles de producir y rígidos, ya que generalmente capturan una instalación, un clima y un conjunto cerrado de fallos y efectos.

Las herramientas de simulación consolidadas modelan el comportamiento sano con gran fidelidad pero tratan la degradación como una corrección agregada.

SOLARIS se sitúa en esta intersección, combinando el rigor físico del modelo de diodo único y las fuentes de datos meteorológicos institucionales con una arquitectura modular de composición de fallos diseñada específicamente para producir datos sintéticos anotados, reproducibles y extensibles.

1.2.1. Detección de fallos en sistemas fotovoltaicos

Las técnicas de detección de fallos fotovoltaicos pueden agruparse en dos grandes corrientes según la naturaleza de la señal de entrada.

La primera se apoya en medidas eléctricas y ambientales sobre las que se aplican desde métodos estadísticos clásicos hasta modelos de aprendizaje automático.

La segunda emplea técnicas de imagen, principalmente termografía infrarroja y electroluminiscencia procesadas mediante arquitecturas de visión por computador como redes convolucionales o detectores basados en *transformers*[1].

En la última década, el campo ha experimentado un desplazamiento claro de los métodos basados en umbrales y reglas en pos de enfoques de aprendizaje automático y profundo. Trabajos recientes reportan precisiones superiores al 90 % en clasificación de fallos empleando desde clasificadores de conjunto como *Random Forest* o *gradient boosting* hasta arquitecturas híbridas más avanzadas[2].

Sin embargo, estas cifras deben interpretarse en contexto. La gran mayoría de los estudios se valida sobre conjuntos de datos pequeños, específicos de

una instalación concreta, y con condiciones experimentales difícilmente reproducibles, lo que dificulta la comparación directa entre métodos y limita las garantías de generalización a instalaciones distintas de la original.

1.2.2. Conjuntos de datos disponibles

La disponibilidad de datos públicos anotados es notablemente escasa en relación con el volumen de literatura sobre algoritmos. En el dominio de series temporales eléctricas, GPVS-Faults[3] ofrece un conjunto de datos experimental recogido en un microgrid fotovoltaico de laboratorio que cubre dieciséis escenarios de fallo. Su principal limitación es estructural, ya que procede de una única instalación física, con una configuración de hardware fija, y los fallos se introducen manualmente a mitad de cada experimento.

En el dominio de imagen, el conjunto PVEL-AD[4] constituye el referente para detección de anomalías en imágenes de electroluminiscencia, con más de 36.000 imágenes de células policristalinas anotadas en diez categorías de defecto. No obstante, trabajos posteriores de *benchmarking* identifican explícitamente la escasez y el desequilibrio de datos como barrera principal para el desarrollo de modelos robustos, llegando a proponer modelos generativos para sintetizar imágenes de defectos con las que remediar dicha carencia[1].

El panorama restante se compone mayoritariamente de conjuntos de datos privados o propietarios, ligados a operadores de plantas concretas y no públicos por motivos comerciales. Esto produce una asimetría notable. Mientras la investigación en algoritmos avanza con rapidez, la disponibilidad de datos sobre la que validarlos y compararlos de forma justa permanece escasa.

1.2.3. Herramientas de simulación fotovoltaica

El ecosistema de simulación fotovoltaica está dominado por herramientas orientadas al estudio de vibilidad y la estimación de producción, con PVsyst, SAM (*System Advisor Model*, de NREL) y PVLib siendo las tres referencias habituales en la literatura comparativa. Las dos primeras son aplicaciones completas dirigidas al diseño de instalaciones, el análisis de pérdidas y la evaluación económica, mientras que PVLib, como se detalla en el Capítulo 2, es una

biblioteca programática que implementa los modelos físicos subyacentes.

Ninguna de estas herramientas, sin embargo, está concebida para la simulación de fallos. Sus modelos de pérdidas agregan la degradación como factores porcentuales globales (por ejemplo, pérdidas anuales por envejecimiento o *soiling* medio mensual), sin capacidad para inyectar modos de fallo individuales con progresión temporal realista ni para generar las anotaciones (etiquetas de fallo, severidad, instante de aparición) que requiere el entrenamiento supervisado. La simulación de fallos en la literatura académica se realiza habitualmente de forma *ad hoc*, mediante modelos construidos en MATLAB/Simulink para cada estudio concreto, sin intención de reutilización ni publicación como herramienta.

1.3. Motivación y Justificación

El mundo sufre una crisis energética[5]. La revolución industrial supuso en su momento una aceleración sin precedentes, propulsando a la humanidad a una nueva era. Una sociedad más avanzada y tecnológica, una sociedad cuya demanda energética, a pesar de la eficiencia alcanzada, crece con el tiempo. Una sociedad insostenible por fuentes tradicionales.

El estado actual está, en mayor o menor medida, condicionado por este progreso acelerado. Datos recientes de la Agencia Internacional de la Energía (IEA) indican que el consumo eléctrico global crecerá al menos 2.5 veces más rápido que la demanda energética total hasta 2030[6]. Las proyecciones de la EIA estadounidense sitúan el incremento del consumo energético mundial cerca del 50 % entre 2018 y 2050[7], mientras que escenarios actuales de la IEA descartan ya un pico próximo en la demanda de combustibles fósiles, contemplando un consumo de petróleo de hasta 113 millones de barriles diarios en 2050[8]. Las implicaciones medioambientales son directas: el sector energético es responsable de alrededor del 60 % de las emisiones globales de gases de efecto invernadero[9].

Hoy en día se habla de una ”nueva revolución industrial”, haciendo alusión a los recientes avances en el campo de la inteligencia artificial por parte de actores como Anthropic, OpenAI, Google o xAI, siendo estos los máximos ex-

ponentes de una larga lista de competidores. Uno de los mayores puntos de fricción en este debate radica en la infraestructura necesaria para soportar y expandir estos modelos de inteligencia y negocio. Los centros de datos son extremadamente costosos en dinero, espacio, materiales de construcción, consumo de agua y, sobretodo, energía.

Los análisis actuales demuestran que el consumo eléctrico global de los centros de datos pasó de aproximadamente 415 TWh en 2024 a una proyección de unos 945 TWh en 2030, siendo ligeramente superior al consumo eléctrico total de Japón hoy en día[10]. La inteligencia artificial es el principal motor de este crecimiento y se estima que los servidores orientados a IA representarán el 44 % del consumo de los centros de datos en 2030, frente al 21 % (proyectado) en 2025[11]. Solo en Estados Unidos, los centros de datos consumirán más electricidad que toda la industria manufacturera energéticamente intensiva combinada (aluminio, acero, cemento, química...) para 2030[10].

Los efectos de semejante demanda energética pueden ser catastróficos. Las consecuencias sobre el medio ambiente del uso de energía no renovable son bien conocidos, y un influjo de centros de datos abastecidos con fuentes contaminantes supondría un incremento adicional de entre 24 y 44 millones de toneladas métricas de CO_2 o equivalentes anuales solo en Estados Unidos para 2030[12]. Esto presionaría aún más unos objetivos climáticos que el propio sector reconoce como cada vez menos alcanzables siguiendo las trayectorias actuales[13].

El planeta no es el único afectado. Se ha observado un aumento en los costes de electricidad pagados por los ciudadanos en zonas donde se han instalado centros de datos[14]. El coste económico del aumento de demanda debido a nuevos centros de datos son, finalmente, asumidos de forma indirecta por ciudadanos ajenos a su operación.

El mundo sufre una crisis energética.

Existe, sin embargo, una posible solución en la energía renovable. La IEA proyecta que más de la mitad del crecimiento de la demanda eléctrica de los centros de datos hasta 2035 será cubierto por fuentes renovables, con un incremento superior a 450 TWh en generación renovable destinada específicamente a este sector[10]. La energía solar fotovoltaica, junto con la eólica, lideran esta transición por su corto plazo de despliegue y su competitividad

económica. A escala global, las renovables ya suministraron el 30 % del consumo eléctrico mundial (en el sector eléctrico) en 2022[9], y constituyen la palanca más viable para desacoplar el crecimiento de la demanda del aumento de emisiones.

Esta orientación no es únicamente técnica o económica, sino también un compromiso institucional. La Agenda 2030 de Naciones Unidas, a través del Objetivo de Desarrollo Sostenible número 7 (ODS 7) "Energía asequible y no contaminante", establece como metas para 2030[9]:

- Garantizar el acceso universal a servicios energéticos asequibles, fiables y modernos.
- Aumentar sustancialmente la proporción de energía renovable en el mix energético mundial.
- Duplicar la tasa global de mejora de la eficiencia energética.
- Impulsar la cooperación internacional para facilitar el acceso a la investigación y tecnología en energías limpias.

El cumplimiento de estos objetivos exige no solo el despliegue masivo de capacidad renovable, sino también la garantía de que dicha capacidad opere de forma fiable, eficiente y económicamente sostenible a lo largo de su vida útil.

El principal factor bloqueante citado en el sector energético es el económico. El dinero se mueve por el camino de menor resistencia, y el análisis de riesgo es uno de los aspectos considerados por los inversores a la hora de tomar decisiones. En el subsector fotovoltaico, la producción depende mayoritariamente del estado de los módulos instalados. Por esta razón, la pronta detección de problemas así como un correcto diagnóstico y mantenimiento de la infraestructura son de suma importancia.

Los algoritmos de detección de fallos en módulos fotovoltaicos son un espacio activo, donde se está invirtiendo y llevando a cabo investigación en nuevas técnicas y tecnologías. Sin embargo, los datos sobre los que diseñar, construir y verificar estos nuevos algoritmos son limitados o propietarios, y no existe una solución de-facto en el espacio.

Dentro de este contexto, se desarrolla SOLARIS con la intención de construir un modelo extensible y modular capaz de simular de forma fehaciente módulos fotovoltaicos y sus posibles fallos, cuyo fin es permitir la verificación de algoritmos existentes y la generación de datos sintéticos anotados para el entrenamiento de nuevos algoritmos basados en machine learning.

1.4. Objetivos de Desarrollo Sostenible

Considerando los objetivos de desarrollo sostenible explícitamente, el proyecto se alinea con los siguientes puntos:

- **ODS 7. Energía asequible y no contaminante:** al facilitar el desarrollo y validación de herramientas que mejoran la fiabilidad y rendimiento de la infraestructura fotovoltaica, SOLARIS contribuye directamente al aumento de la proporción de energía renovable en el mix energético global (meta 7.2) y a la mejora de la eficiencia energética del sector (meta 7.3).
- **ODS 9. Industria, innovación e infraestructura:** el proyecto fomenta la innovación en herramientas de simulación y diagnóstico, apoyando el desarrollo de infraestructuras energéticas resilientes y sostenibles (meta 9.4).
- **ODS 12. Producción y consumo responsables:** mediante la prolongación de la vida útil y la optimización del rendimiento de los activos fotovoltaicos, se contribuye a una gestión más eficiente de los recursos naturales (meta 12.2).
- **ODS 13. Acción por el clima:** al apoyar indirectamente la viabilidad económica de la generación renovable frente a fuentes fósiles, SOLARIS contribuye a las medidas de mitigación del cambio climático (meta 13.2).

1.5. Objetivos

Los objetivos del proyecto SOLARIS se pueden resumir en una oración: diseñar y construir un sistema modular fácilmente extensible que permita simular módulos fotovoltaicos modelando degradaciones y fallos a lo largo del tiempo.

De forma específica, el proyecto SOLARIS busca:

- **Generación de datos sintéticos a partir de principios físicos:** El objetivo principal del sistema es la generación de datos de módulos solares y sus modos de fallo y degradación a través de una simulación fiel a la realidad basada en principios físicos. La herramienta debe permitir la generación rápida de datos usables que experimentalmente tomarían décadas.
- **Modularidad:** Uno de los requisitos principales del sistema es la modularidad. La modularidad permite configurar la herramienta y adaptarla a las necesidades del contexto en el que se emplea. Esto debe incluir parámetros de la simulación así como permitir la inclusión o exclusión de los distintos modos de degradación o fallos que puede experimentar un módulo fotovoltaico a lo largo de su vida.
- **Extensibilidad:** El sistema no solo debe permitir sino facilitar su extensión. Esto permite la implementación de nuevos y mejores modelos de degradación y fallos, así como mejoras al sistema central. El sistema debe ser un producto fácilmente moldeable, capaz de crecer con las necesidades del contexto en el que se emplea.
- **Versatilidad:** Aunque en gran parte este objetivo es una consecuencia de la correcta ejecución de los anteriores tres, merece una mención explícita. El sistema no debe estar limitado a una sola aplicación. Aunque el documento se centra principalmente en la generación de datos para desarrollo y validación de algoritmos, la herramienta es adaptable a una amplia variedad de problemas, tales como la validación de modelos teóricos de fallos o degradación, el estudio de viabilidad en zonas específicas, o estudio teórico de rendimiento de nuevos módulos fotovoltaicos.
- **Rendimiento, compatibilidad y escalabilidad:** El sistema debe tener un rendimiento adecuado, y aplicar técnicas de aceleración como el para-

lelismo cuando sea apropiado. A su vez ha de ser ejecutable en una amplia variedad de hardware, permitiendo el aprovechamiento de este a través de configuración del usuario. Debido a la naturaleza temporal de los modelos de efecto de los módulos fotovoltaicos, su simulación resulta costosa en cómputo.

- **Robustez:** El sistema ha de ser robusto ante fallos de configuración y errores que puedan aparecer durante la ejecución. El sistema debe validar la configuración suministrada y avisar de errores de forma clara y lo más anticipada posible, evitando largas esperas sin resultados útiles. A su vez, debe ser resistente a fallos durante la simulación, bien sanando errores o continuando la ejecución marcando los problemas encontrados.

2. Conocimiento Teórico y Técnico

2.1. Módulos fotovoltaicos

La energía solar es una de las principales fuentes de energía renovable conocidas. Los módulos fotovoltaicos, también conocidos como paneles solares, son componentes eléctricos capaces de convertir energía lumínica en corriente eléctrica.

Sus características específicas dependen de diversos factores como pueden ser tamaño o materiales de construcción, pero generalmente se componen de distintas capas envolventes a una junta N-P, que excitada por contacto por fotones produce una corriente eléctrica a través de sus polos.

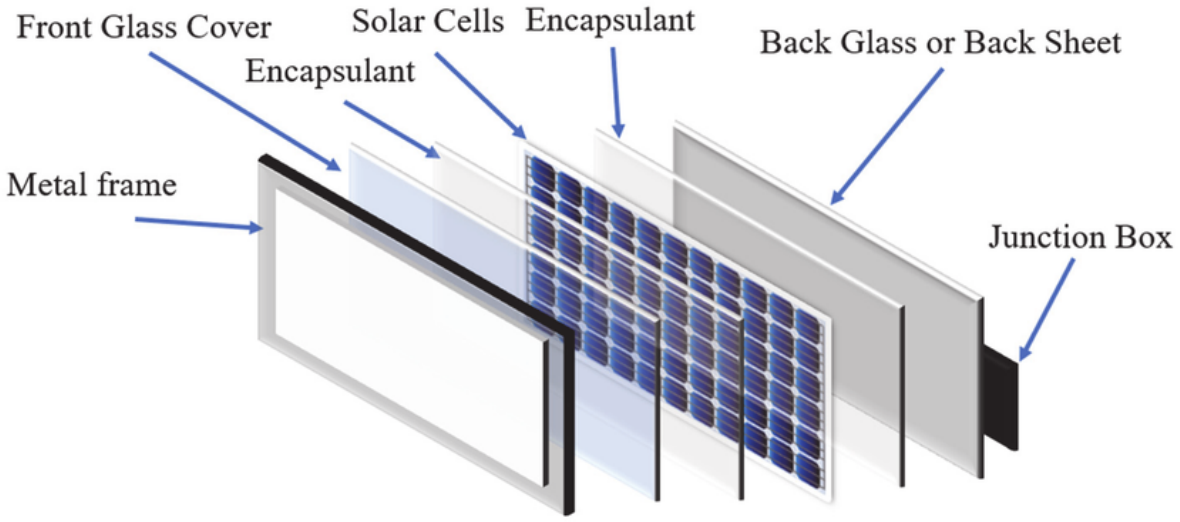


Figura 2.1. Diagrama explosionado de un módulo fotovoltaico [15].

Debido a su salida en corriente directa, suelen operarse de forma conjunta,

agrupándose en hileras (strings) de módulos conectados a un conversor DC-AC. Cuando estas hileras se disponen en conjunto, se les denomina granjas solares, que son instalaciones individuales o industriales capaces de suministrar energía a la red eléctrica.

2.1.1. Curva I-V

El comportamiento eléctrico de un módulo fotovoltaico se caracteriza mediante su curva corriente-tensión (I-V), que describe la corriente de salida para cada nivel de tensión aplicado bajo unas condiciones de irradiancia y temperatura específicas. Estas normalmente se fijan bajo un estándar conocido como STC o *Standard Tests Conditions*, que fijan la irradiancia a $1000\frac{W}{m^2}$ y la temperatura a $25°$C. De esta forma, se puede comparar el funcionamiento de distintos módulos bajo las mismas condiciones.

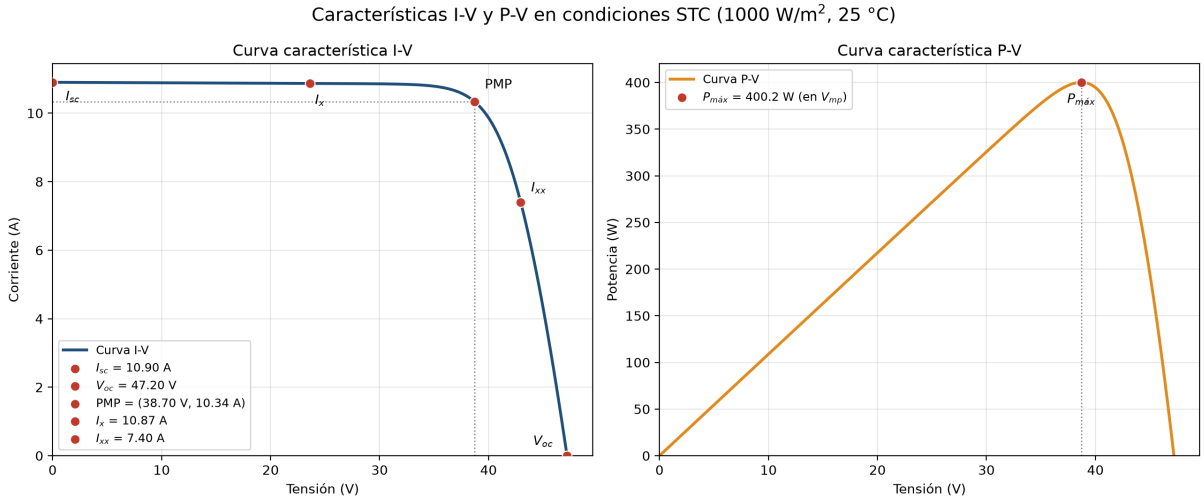


Figura 2.2. Curva I-V de un módulo comercial común bajo condiciones estándar.

La curva I-V se define por tres puntos principales, conocidos como puntos de operación característicos:

- I_{sc}: La corriente de cortocircuito, es la corriente máxima que el módulo puede producir dado un voltaje nulo entre sus terminales.
- V_{oc}: Tensión (voltaje) máxima entre los terminales del módulo en circuito abierto, es decir, con corriente cero.

- MPP o PMP: El punto de operación ideal del módulo situado entre los dos anteriores, *Maximum Power Point* donde el producto entre la corriente y la tensión del módulo, en otras palabras, la potencia, es máxima.

En algunos casos también se incluyen los puntos I_x e I_{xx}, que permiten la definición de la curva I-V de forma precisa manteniendo el número de puntos definidos a cinco. En concreto, I_x se define como el valor de la corriente en $½V_{oc}$, el punto medio entre voltaje máximo y nulo. A su vez, I_{xx} es el valor de la corriente cuando el voltaje es $½(V_{mp} + V_{oc})$ donde V_{mp} es el valor del voltaje en el PMP.

Los sistemas fotovoltaicos comerciales implementan algoritmos de seguimiento del punto de máxima potencia (*MPP Tracking* o simplemente *MPPT*) para operar el módulo lo más cerca posible de este punto en todo momento, dado que su posición sobre la curva depende de las condiciones ambientales como la irradiancia y temperatura.

La forma de la curva I-V, y en particular la posición y pronunciación de la caída cercana al *PMP*, es altamente sensible a la presencia de fallos o degradación en el módulo. Por esta razón es una de las señales más directas e informativas para la detección de fallos, y por ende la salida fundamental que cualquier simulador de módulos fotovoltaicos debe ser capaz de producir con fidelidad.

2.1.2. Circuito equivalente: diodo único

Para modelar matemáticamente el comportamiento eléctrico de un módulo fotovoltaico, generalmente se recurre a circuitos equivalentes que reproducen su curva I-V mediante combinaciones de componentes eléctricos ideales, simplificando así su análisis y tratamiento. El modelo más extendido para estos módulos es el circuito equivalente de diodo único (*Single-Diode Model* o SDM)[16]. En este modelo, el módulo fotovoltaico se reduce a:

- I_l: Corriente fotogenerada (Light Current), se representa como una fuente de corriente ideal fotoactivada, es decir, cuya salida depende de la incidencia de luz.

- D: Un diodo conectado en paralelo con la fuente I_l, modela la unión P-N del módulo.
- R_{sh}: Resistencia shunt, que modela pérdidas de corriente por impurezas en los materiales o errores de construcción.
- R_s: Resistencia en serie, que representa la resistencia interna del módulo en sus contactos y conexiones internas.

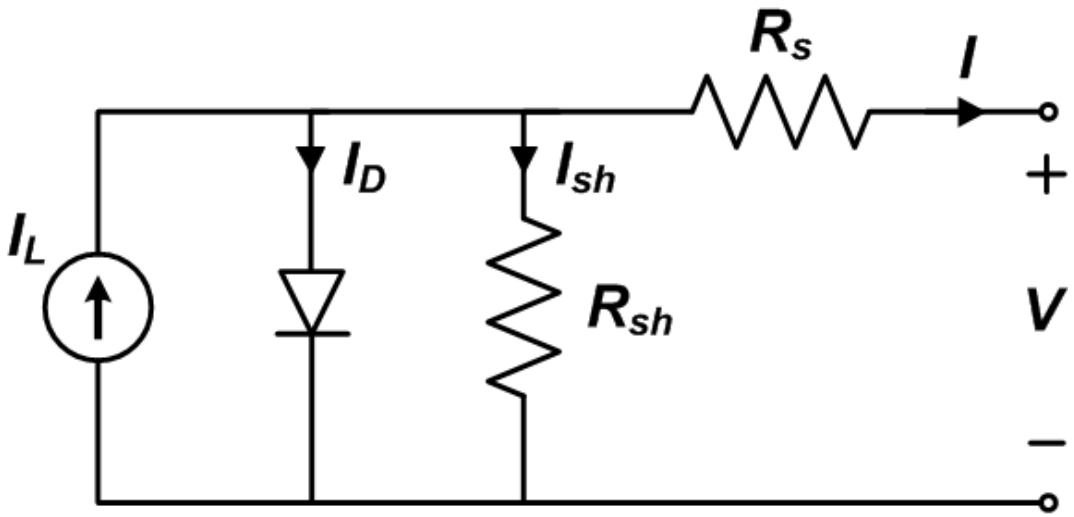


Figura 2.3. Diagrama de circuito equivalente SDM[16].

En la figura 2.3 se pueden observar los componentes ideales que determinan el comportamiento del circuito. A partir de este circuito equivalente, se puede construir la ecuación implícita que produce la curva I-V correspondiente[16], y por lo tanto la del módulo fotovoltaico:

$$I = I_L - I_0 \left[\exp\left(\frac{V + IR_s}{nV_T} \right) - 1 \right] - \frac{V + IR_s}{R_{sh}}$$

Bajo unas condiciones determinadas de irradiancia y temperatura, los elementos corriente fotogenerada (I_L), corriente de saturación inversa del diodo (I_0), resistencia serie (R_s), resistencia shunt (R_{sh}), y el producto térmico del diodo (nN_sV_T, que combina el factor de idealidad del diodo n, el número de células en serie N_s, y la tensión térmica V_T) son suficientes para reconstruir la curva I-V completa de un módulo.

La popularidad del SDM frente a modelos alternativos como el modelo de Sandia[17], de naturaleza más empírica; o el modelo de dos diodos, que permite mucho mayor precisión; radica en que sus parámetros tienen una interpretación física directa a la vez que un coste computacional relativamente modesto, lo cual resulta especialmente conveniente a efectos de simulación de fallos. Perturbar un parámetro físico concreto permite modelar de forma trazable el efecto de un mecanismo de degradación específico, en lugar de ajustar coeficientes empíricos sin correspondencia física clara.

Para una derivación completa del modelo se remite a De Soto et al.[18], referencia fundacional en la que se apoya buena parte de la literatura relacionada con el SDM.

2.1.3. Base de datos CEC

La aplicación práctica del modelo de diodo único requiere los cinco parámetros del *SDM* (I_L, I_0, R_s, R_{sh}, nN_sV_{th}) para el módulo concreto que se desea simular. Obtener estos parámetros mediante caracterización experimental es costoso y poco escalable, por lo que en la práctica se recurre a bases de datos que contienen datos de referencia a partir de las especificaciones de los fabricantes que permiten el cálculo de los cinco parámetros *SDM*.

La base de datos del California Energy Commission[19] (CEC)es una de las más utilizadas tanto en investigación como en herramientas de software de referencia del sector a estos efectos, como `pvlib`. Esta base de datos recopila los parámetros del SDM para miles de modelos comerciales de módulos fotovoltaicos, ajustados de forma que reproduzcan fielmente las curvas I-V especificadas por el fabricante bajo condiciones estándar de prueba (STC).

Es importante señalar que los parámetros recogidos en la base de datos CEC reflejan el estado del módulo tras el proceso de estabilización inicial durante la degradación inducida por luz (LID). Es decir, corresponden a un punto de partida ya parcialmente degradado respecto al módulo recién fabricado, y no a un módulo en su estado teórico ”perfecto”. Esta consideración resulta relevante a la hora de interpretar los parámetros como base sobre la que posteriormente se inyectan fallos adicionales, ya que no representa un punto cero absoluto de degradación, sino un estado de referencia ya afectado por

uno de los mecanismos de degradación más comunes y documentados.

2.1.4. Efectos: Fallos y Modos de Degradación

Los módulos fotovoltaicos, a pesar de su naturaleza estática y la ausencia de partes móviles, están sujetos a múltiples mecanismos de fallo y degradación a lo largo de su vida útil. Estos mecanismos tienen orígenes muy diversos, desde defectos de fabricación hasta condiciones ambientales extremas o pérdida de estanqueidad del encapsulado, y se manifiestan con distinta severidad y velocidad de progresión.

La literatura especializada ha dedicado un esfuerzo considerable a catalogar y caracterizar estos modos de fallo. El informe IEA-PVPS T13-01 de Köntges et al.[20] constituye una de las referencias más exhaustivas en este sentido, estableciendo una taxonomía de fallos ampliamente adoptada por la comunidad. Aghaei et al.[21] ofrecen una revisión actualizada de cada tipo de fallo, mientras que el trabajo de Jordan y Kurtz[22] se centra específicamente en las tasas de degradación observadas experimentalmente a largo plazo. Las fichas técnicas PVFS del informe IEA-PVPS T13-30:2025[23] han consolidado y actualizado esta clasificación con datos de campo de instalaciones recientes.

Conviene distinguir tres conceptos que la abstracción ”efecto” encapsula:

- Un **efecto** es cualquier alteración de la respuesta de un panel respecto a su comportamiento nominal.
- Un **fallo** es un efecto de naturaleza condicional, un modo de avería que puede o no manifestarse, con un instante de aparición gobernado por una distribución estadística y los parámetros de entorno.
- Una **degradación** es un efecto gradual y siempre presente, que comienza desde el primer instante de vida del módulo.

Independientemente de esta clasificación temporal, todo efecto termina por manifestarse eléctricamente a través de uno de dos canales de entrada al modelo de diodo único descrito en la sección anterior, bien la irradiancia efectiva que alcanza la célula o su temperatura de operación, o bien los cinco parámetros físicos del circuito equivalente (I_L, I_0, R_s, R_{sh}, nN_sV_{th}). A grandes rasgos, los

fallos fotovoltaicos pueden agruparse en dos grandes familias según el mecanismo físico subyacente y su efecto sobre el comportamiento eléctrico del módulo, si bien existen casos en los que un mismo tipo de fallo puede manifestarse en ambos niveles simultáneamente, como ocurre con las fisuras.

Ciertos mecanismos no alteran el módulo en si, sino la cantidad de luz efectiva que llega a incidir sobre él. El *soiling* o acumulación de polvo, polen u otros residuos sobre la superficie, y el sombreado parcial son los ejemplos más directos. Ambos actúan como un factor de atenuación aplicado sobre la **irradiancia** de entrada antes de que esta alcance el modelo eléctrico, sin necesidad de modificar ninguno de los cinco parámetros del *SDM*. La degradación óptica del vidrio opera de forma análoga, aunque de manera permanente en lugar de estacional o reversible como ocurre con el *soiling*.

De forma menos evidente, algunos efectos alteran la respuesta del módulo no a través de la irradiancia ni de los parámetros eléctricos directamente, sino modificando la temperatura de operación de la célula, que a su vez es una de las dos entradas de `calcparams_cec`. La degradación del encapsulante o del *backsheet*, por ejemplo, puede reducir la capacidad de disipación térmica del módulo, elevando su temperatura de operación por encima de la esperada para unas condiciones ambientales dadas, y con ello desplazando el punto de funcionamiento del modelo eléctrico sin que ningún parámetro del SDM haya sido alterado explícitamente.

La mayoría de los mecanismos de fallo y degradación documentados en la literatura[20], [21], [23] actúan directamente sobre los cinco parámetros físicos del circuito equivalente, habitualmente incrementando o reduciendo uno o varios de ellos de forma simultánea:

- La **corrosión de contactos** y el **deterioro de las interconexiones** incrementan la resistencia serie (R_s), al degradar progresivamente las rutas de conducción entre células.
- La **degradación inducida por potencial** (PID) y ciertas formas de humedad interna reducen la resistencia shunt (R_{sh}), causando pérdidas de corriente adicionales dentro de la célula.
- La **degradación inducida por luz** (LID), ya mencionada como estado base de la propia base de datos CEC, incrementa la corriente de saturación

inversa (I_0) y reduce ligeramente I_L, disminuyendo la eficiencia de conversión de la célula.

- Las **microfisuras** incrementan R_s de forma localizada, al interrumpir parcialmente las rutas de conducción interna. Según su extensión y ubicación pueden además reducir I_L si llegan a aislar eléctricamente una fracción relevante del área activa de la célula.

Es importante destacar que estas vías no son mutuamente excluyentes para un mismo mecanismo físico, tal y como se ha mencionado previamente que ocurre en el caso de las fisuras. Un mismo efecto puede requerir inyectarse en ambos puntos del pipeline simultáneamente para representarse con fidelidad. Esta doble naturaleza es, de hecho, uno de los factores que motivó el diseño de SOLARIS como una arquitectura de bloques de efectos independientes y componibles, en lugar de una simulación monolítica con un único punto de inyección de fallos.

2.2. Paralelización

La técnica de paralelización en el contexto del desarrollo de software se define como una división de operaciones computacionales en subtareas que pueden ser ejecutadas simultáneamente en distintos núcleos o máquinas. Con esto se pretende mejorar el rendimiento y la eficiencia del sistema así como aprovechar en la medida de lo posible el hardware disponible, reduciendo en gran parte el tiempo total necesario para completar una tarea.

La paralelización es una práctica esencial en tareas computacionalmente intensivas, y es especialmente prevalente en aplicaciones que manejan grandes volúmenes de datos o requieren cálculos intensivos e independientes.

La paralelización permite una ejecución más rápida de tareas y el procesamiento eficiente de grandes datasets, y aumenta la escalabilidad de diversas aplicaciones. Sin embargo, también presenta desafíos significativos, como la gestión de la sincronización entre tareas paralelas, el balanceo de carga para asegurar que los recursos se utilicen de manera equitativa, y la minimización de la sobrecarga de comunicación entre los distintos procesos.

Podemos diferenciar dos tipos principales de paralelización:

- **Paralelización de datos:** Consiste en dividir el conjunto de datos en partes más pequeñas que pueden ser procesadas en paralelo por procesos individuales. Este enfoque es común en aplicaciones de análisis de datos, donde se pueden realizar operaciones similares sobre subconjuntos de datos de manera concurrente.
- **Paralelización de tareas:** Se refiere a la ejecución simultánea de diferentes tareas o funciones independientes que son parte de un mismo proceso. Este método es útil cuando las tareas tienen dependencias nulas o limitadas entre sí y pueden progresar en paralelo sin esperar unas a otras, o en disposiciones similares a la segmentación de instrucciones de un procesador.

La tarea de simulación de módulos solares se sitúa firmemente en la primera categoría, debido a que es un conjunto de operaciones independientes idéntico ejecutado sobre un gran volumen de datos.

2.2.1. Paralelización en Python

Python posee una particularidad relevante para la paralelización dada su arquitectura interpretada, el Global Interpreter Lock (GIL). El GIL es un mecanismo de Python que, en un momento dado, garantiza que solo un hilo (*thread*) pueda ejecutar bytecode de Python dentro de un mismo proceso. Esta restricción simplifica la gestión de memoria del intérprete y evita condiciones de carrera internas. A su vez y como consecuencia, impide al módulo `threading` de la biblioteca estándar alcanzar paralelismo real en tareas limitadas por CPU (*CPU-bound*). Es el caso de las simulaciones fotovoltaicas, donde múltiples hilos dentro de un mismo proceso Python se ejecutarían de forma secuencial sobre un único núcleo sin beneficio alguno de rendimiento, e incluso incurriendo una ligera penalización debido al coste de cambio de contexto entre hilos.

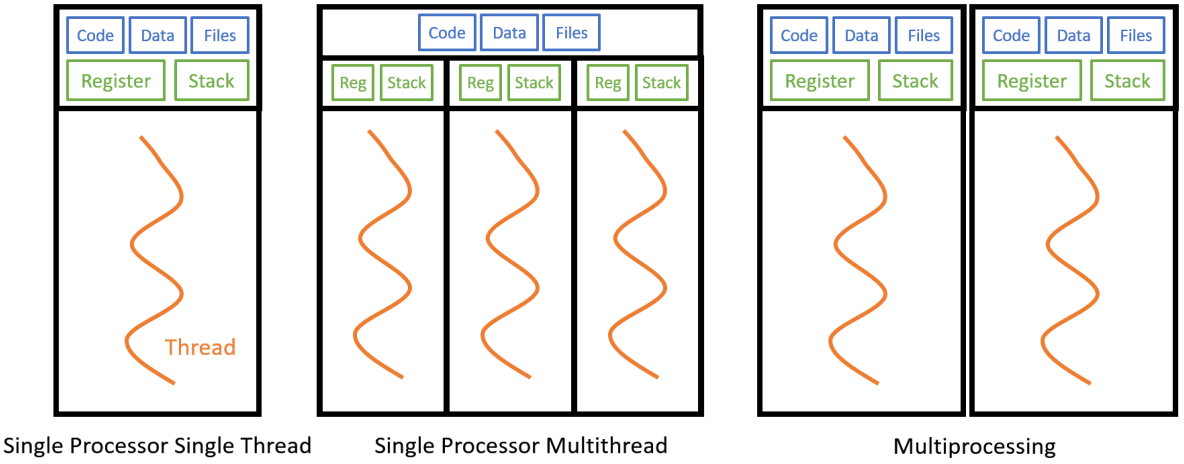


Figura 2.4. Esquema diferenciativo entre múltiples hilos y múltiples procesos[24].

A cambio, Python ofrece el módulo `multiprocessing`, que permite paralelismo real distribuyendo el trabajo empleando múltiples procesos independientes en lugar de hilos. Cada proceso generado cuenta con su propio intérprete de Python y su propio GIL, lo que elimina la condición limitante descrita y permite el uso efectivo de los múltiples núcleos disponibles en la máquina. Generar múltiples procesos acarrea un mayor coste de creación en comparación con el uso de hilos, y conlleva la necesidad de serializar (*pickle*) los datos que se transfieren entre el proceso principal y los procesos secundarios, ya que los procesos no comparten memoria de forma nativa como se puede ver en la figura 2.4.

De la librería `multiprocessing`, la abstracción `Pool` resulta especialmente adecuada para SOLARIS. Un objeto `Pool` gestiona un conjunto fijo de procesos trabajadores (*workers*) y permite distribuir automáticamente un conjunto de tareas independientes entre ellos mediante métodos como `map` o `imap`, encargándose del reparto de trabajo y la recolección de resultados. Dada la independencia computacional entre instancias de módulos fotovoltaicos simulados, el conjunto total a generar puede repartirse directamente entre los procesos del `Pool`, sin necesidad de sincronización ni comunicación durante su ejecución.

2.3. Herramientas y Librerías

A continuación se listan y explican brevemente las librerías y herramientas de las que depende el proyecto SOLARIS.

2.3.1. PVLib

PVLib es la herramienta central sobre la que se construye el proyecto SOLARIS.

Se trata de una biblioteca de Python de código abierto especializada en el modelado del rendimiento de sistemas fotovoltaicos. Sus orígenes se remontan a 2013, cuando nació como una traducción a Python de la herramienta para MatLab con la que comparte nombre. Originalmente desarrollada por Sandia National Laboratories[25], el proyecto ha crecido considerablemente, contando en la actualidad con contribuciones de más de un centenar de desarrolladores e investigadores de todo el mundo, y es mantenido de forma colaborativa por un grupo de referencia en modelado fotovoltaico procedente de instituciones como Sandia, NREL, DNV y diversas universidades. PVLib funciona como estándar en la comunidad de investigación fotovoltaica, proporcionando implementaciones abiertas, validadas y reproducibles de los modelos y algoritmos más relevantes del sector.

La biblioteca ofrece funciones para prácticamente todas las etapas del modelado de un sistema fotovoltaico. Adicionalmente proporciona acceso directo a la base de datos CEC, así como a otras bases de datos de parámetros de módulos e inversores, gestionando la descarga, el análisis sintáctico (*parsing*) y el mantenimiento de estos conjuntos de datos[26]. La elección de PVLib responde tanto a su estatus como referencia validada por la comunidad científica como a la disponibilidad directa de funciones necesarias para implementar el modelo de diodo único sin necesidad de reimplementar desde cero la física subyacente.

2.3.2. PVGIS

PVGIS (*Photovoltaic Geographical Information System*) es una herramienta de acceso libre desarrollada y mantenida por el *Joint Research Centre* (JRC) de la Comisión Europea[27], cuya función es proporcionar datos solares y estimaciones del rendimiento de sistemas fotovoltaicos combinando bases de datos de irradiancia derivadas de observación satelital y de análisis meteorológico. Su cobertura abarca todo el globo exceptuando los polos Norte y Sur[28].

PVLib incorpora dentro de su subpaquete `iotools` funciones específicas para conectar con la API de PVGIS de forma directa[26], recuperando datos en un formato compatible con el resto de la librería.

El uso de PVGIS se favorece debido a, por un lado, su carácter público, gratuito y de origen institucional, lo que garantiza la reproducibilidad sin depender de servicios comerciales con acceso restringido o de pago; por otro, la disponibilidad de series temporales históricas de varios años de duración para una misma ubicación, lo cual permite generar conjuntos de datos sintéticos representativos de la variabilidad real de las condiciones de operación a las que se ve sometido un módulo fotovoltaico a lo largo de su vida útil; y finalmente, la compatibilidad *Out Of The Box* de sus datos con las funciones internas de PVLib.

2.3.3. Open-Meteo

Open-Meteo es una API meteorológica de código abierto y acceso gratuito para uso no comercial, que agrega datos de múltiples modelos numéricos de predicción meteorológica procedentes de servicios como ECMWF, NOAA, DWD o Météo-France[29]. Además de sus capacidades de predicción, Open-Meteo ofrece acceso a datos históricos a través de ERA5, con cobertura global, resolución horaria y sin huecos temporales desde 1940 hasta la actualidad[30].

Si bien PVGIS constituye la fuente principal de irradiancia y posición solar, Open-Meteo se emplea como fuente encargada específicamente de suministrar variables atmosféricas no fotovoltaicas que resultan relevantes, espe-

cialmente para futuras implementaciones de modelos de degradación, fallo y efectos, permitiendo un desarrollo más cómodo. Tan solo han de añadirse los campos deseados a la llamada dentro del fichero `meteo.py`.

2.3.4. NumPy

NumPy es la biblioteca fundamental para la computación numérica en Python[31]. Proporciona el tipo de dato `ndarray`, una estructura n-dimensional homogénea sobre la que se definen operaciones vectorizadas que se ejecutan en código compilado, evitando los bucles explícitos del intérprete de Python y alcanzando un rendimiento más próximo al de lenguajes de bajo nivel.

Constituye la base sobre la que se construye la práctica totalidad del ecosistema científico de Python, incluyendo tanto `Pandas` como `PVLib`, las principales dependencias del proyecto SOLARIS.

2.3.5. `pandas`

`pandas` es una librería de código abierto para la manipulación y el análisis de datos tabulares en Python[32][33]. Su estructura central es el `DataFrame`, una tabla bidimensional de columnas etiquetadas indexada por filas, que ofrece operaciones de alto nivel para el filtrado, la agregación, la alineación y la transformación de datos. Está construida sobre NumPy, de donde hereda la eficiencia de las operaciones vectorizadas, y se ha consolidado como estándar para el tratamiento de datos en el ecosistema de Python.

En el contexto de SOLARIS, pandas actúa como la estructura de datos de-facto a lo largo de todo el pipeline. Los datos tanto de PVGIS como Open-Meteo se convierten a formato `DataFrame` indexado temporalmente, lo que permite la alineación directa de ambas fuentes mediante operaciones nativas de `pandas` antes de su consumo por `PVLib`. Del mismo modo, los resultados de la simulación y sidecar de atribución se estructuran y serializan a disco en formato Parquet a través de pandas, preservando el índice temporal y los tipos de cada columna. Esta centralidad de pandas en el diseño del pipeline responde tanto a su profunda integración con el resto de dependencias del proyecto como a la expresividad que ofrece para la inspección y depuración

de datos durante el desarrollo y ejecución.

2.3.6. Parquet & PyArrow

Apache Parquet es un formato columnar de datos en disco de código abierto y diseñado para el almacenamiento y consulta eficientes de grandes volúmenes de datos tabulares[34]. Parquet agrupa los valores por columna, permitiendo esquemas de compresión y codificación más efectivos sobre datos homogéneos que los permitidos por formatos orientados a filas como puede ser CSV. Adicionalmente, el formato conserva el esquema y los tipos de dato de forma explícita, evitando la pérdida de información de tipado, así como la lectura selectiva de columnas.

PyArrow es la implementación en Python de Apache Arrow, un proyecto de la Apache Software Foundation que proporciona el motor de lectura y escritura sobre el que se apoyan bibliotecas como Pandas para operar con ficheros Parquet[35]. PyArrow actúa como el motor subyacente invocado de forma transparente por Pandas al serializar los resultados de la simulación.

El uso de Parquet se favorece debido a varios motivos. La naturaleza de los datos generados por SOLARIS (series temporales numéricas, amplias y homogéneas) se ajusta al formato columnar que Parquet permite comprimir, reduciendo el tamaño en disco frente a alternativas. La conservación del esquema de tipado garantiza la inmutabilidad de los tipos de dato durante ciclos de escritura y lectura. Por último, la estructura final como conjunto de datos particionado en múltiples ficheros encaja directamente con la estrategia de fragmentación y paralelización del proyecto donde cada *batch* de paneles (unidad de trabajo manejada por procesos paralelos) se escribe como un fichero independiente dentro de un directorio de esquema único, que un consumidor puede leer en su totalidad o de forma parcial.

2.3.7. Otros

TOML

`TOML` (*Tom's Obvious Minimal Language*) es un formato de fichero de configuración de sintaxis mínima, diseñado para ser legible por humanos y de correspondencia directa con estructuras de datos de Python[36].

SOLARIS centraliza toda su configuración en un único fichero `TOML`, leído mediante el módulo `tomllib` de la librería estándar de Python (con *fallback* a `tomli` en versiones anteriores a la 3.11). De este modo, los parámetros de cada ejecución quedan descritos de forma declarativa y reproducible, sin necesidad de modificar el código fuente.

`argparse`

Para permitir la ejecución de un programa o herramienta con distintas opciones directamente desde la consola de comandos a través del uso de *flags*, Python implementa el módulo `argparse`, capaz de interpretar llamadas al código desde la línea de comandos[37].

SOLARIS hace uso de *flags* principalmente en las funcionalidades *helper* para la adquisición de datos meteorológicos, y también en menor medida en la ejecución de la simulación principal.

El argumento `--cfg path/to/namefile.toml` indica el archivo de configuración a usar, mientras que las funcionalidades meteorológicas definen su comportamiento principalmente a través de los *flags* empleados en su llamada.

`requests`

Python permite realizar peticiones HTTP de forma sencilla y con implementaciones de alto nivel a través de la librería `requests`[38].

En SOLARIS se emplea para consultar la API de archivo de Open-Meteo, re-

cuperando las variables atmosféricas históricas en formato JSON que posteriormente se estructuran en un `DataFrame`.

`tqdm`

La librería `tqdm` añade barras de progreso a procesos iterativos en Python con una sobrecarga mínima[39].

Durante la simulación que ejecuta SOLARIS, se utiliza para mostrar el avance sobre el conjunto de paneles procesados, integrándose con el sistema de `logs` del proyecto de modo que los mensajes informativos no interfieran con la representación de la barra.

3. Descripción del Sistema

Solaris es una herramienta de simulación de paneles y granjas solares.

La arquitectura modular del proyecto basada en un flujo por etapas facilita su extensión y modificación para adaptarse a las necesidades de cualquier usuario.

Todo el código producido puede encontrarse en `https://github.com/MewKitBit/SOLARIS`

3.1. Arquitectura del proyecto

SOLARIS se puede entender como una *pipeline* de datos, que comienza con unos datos de entrada, realiza una serie de cálculos, y finalmente devuelve un resultado.

Los datos de entrada se pueden definir como:

- **Datos de entorno:** Son los datos obtenidos desde `PVGIS` u `Open-Meteo`. Representan el entorno de los módulos fotovoltaicos a simular, influenciando su potencia producida y la aparición y progresión de efectos, fallos y modos de degradación.
- **Definición CEC del módulo:** Se trata de los parámetros CEC que categorizan el módulo fotovoltaico a simular.
- **Configuración de la simulación:** Se deben definir los valores de configuración de la simulación, tales como el límite de memoria RAM a utilizar o el número de procesos ejecutándose a la vez.
- **Modelos de efecto, fallo y degradación:** El usuario debe implementar, haciendo uso de la clase abstracta definida para este fin, todos los mo-

delos de efecto, fallo y degradación que pueden afectar a un módulo fotovoltaico y se deseen simular.

El archivo `main.py` contiene las funciones que dirigen la simulación a través de sus distintas etapas, moviendo los datos entre las funciones de los archivos `effect_generator.py` y `data_generator.py`, y repartiendo trabajo y gestionando los procesos generados para las tareas paralelizables.

La salida final se compone de dos directorios parquet, que son `observables` y `attribution`. `observables` contiene la serie temporal de cada panel simulado, mientras que `attribution` guarda un *sidecar* de atribuciones donde se recoge el momento inicial de cada efecto o fallo aplicado por panel.

3.2. Diagrama de flujo de datos

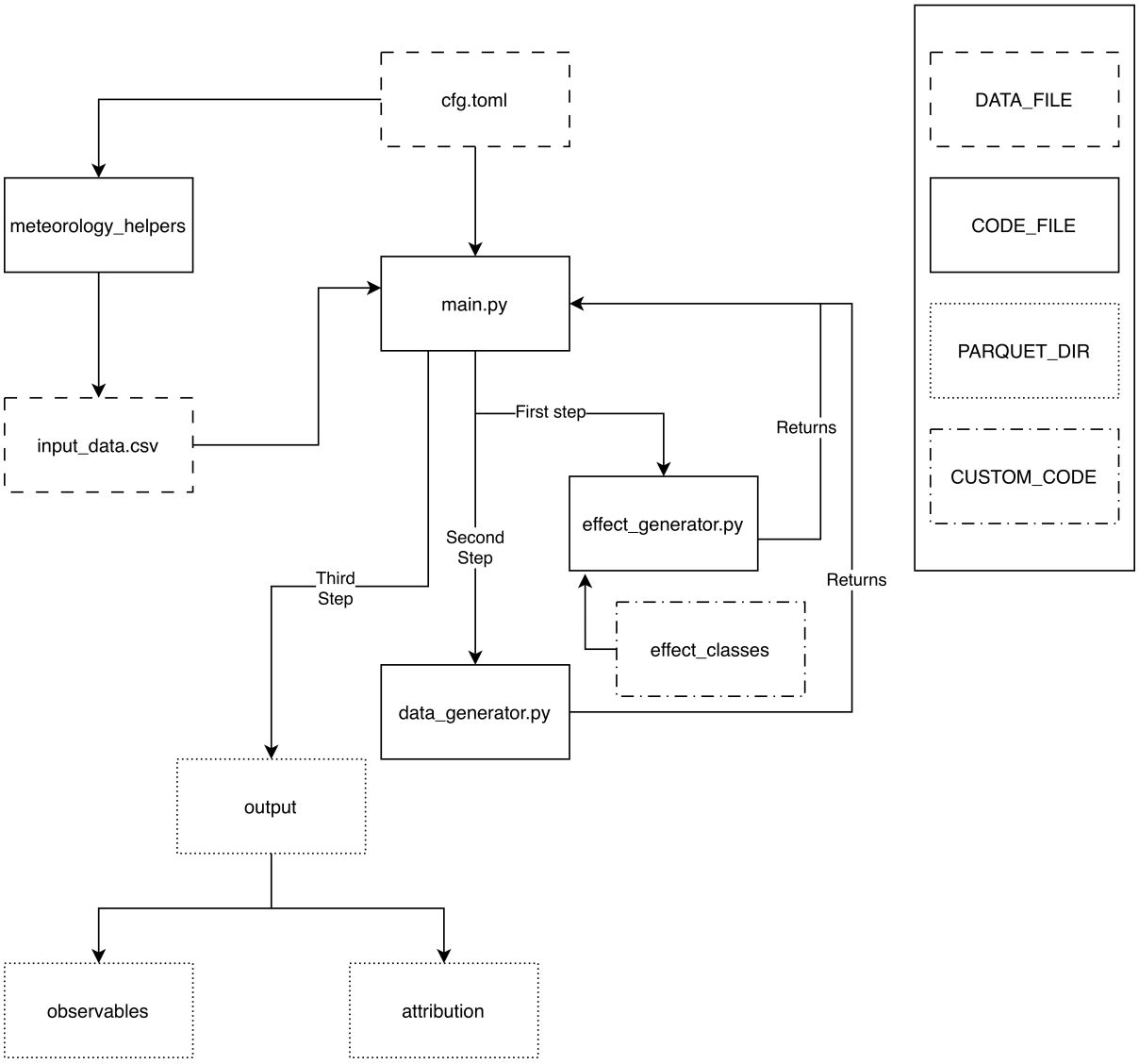


Figura 3.1. Diagrama de flujo de datos de SOLARIS.

En el diagrama 3.1 se muestra la secuencia de ejecución habitual de SOLARIS, incluyendo la adquisición de datos a través de los *helpers* incluidos en el repositorio.

Existen dos puntos de entrada cuya implementación es necesaria para el uso de SOLARIS, con un tercer punto de entrada opcional:

- El archivo de configuración `.toml` se debe definir siguiendo las reglas delineadas en este para configurar la simulación.
- Se deben definir las clases que implementan los modelos de efecto, fallo y degradación que se quieren incluir en la simulación para la posterior consumición en la clase `effect_generator.py`.
- Opcionalmente, se puede prescindir del uso de los *helpers* meteorológicos, suministrando datos de otras fuentes siempre que su formato sea conforme a lo esperado por la herramienta SOLARIS.

Los datos de configuración del archivo `.toml` se usan en los *helpers* meteorologicos para obtener los datos de entrada `input_data.csv`.

Posteriormente tanto el archivo de configuración como los datos suministrados en `input_data.csv` se usan por la función principal `orchestrator( ... )` de SOLARIS para llevar a cabo el proceso completo de simulación.

La salida final se ubica en el directorio configurado, donde se crean los dos directorios contenedores de archivos parquet que se van escribiendo según los *workers* instanciados terminan de procesar los *chunks* asignados, siendo cada archivo parquet uno de estos *chunks*.

3.3. Configuración y Ejecución

El funcionamiento de SOLARIS se basa en dos modos de ejecución desacoplados. Una primera fase de adquisición de datos recopila las series temporales de entrada de las fuentes externas y las consolida en disco. Una segunda fase de simulación consume dichas series para generar el conjunto de datos sintético de salida.

Ambas fases se definen desde un único fichero de configuración en formato TOML, que actúa como pilar de la simulación y describe la totalidad de los parámetros de una ejecución. De este modo, reproducir o variar un experimento se reduce a editar dicho fichero, sin necesidad de modificar el código fuente.

3.3.1. El fichero de configuración unificado

El fichero `.toml` se organiza en secciones que agrupan los parámetros del sistema en categorías concretas. Su carácter unificado es deliberado, tanto la herramienta de adquisición como el motor de simulación leen el mismo fichero, garantizando que los datos descargados y la simulación que los consume comparten exactamente la misma ubicación, periodo y configuración del panel.

Ubicación y periodo (`[location]`, `[period]`).

Definen el punto geográfico especificando latitud, longitud y zona horaria, y el intervalo temporal inclusivo sobre el que se realizan las llamadas a `Open-Meteo` y PVGIS.

```toml
[location]
latitude  = 43.4636 # decimal degrees, positive = north
longitude = 3.8226 # decimal degrees, positive = east
timezone  = "UTC" # IANA timezone; localises the gathered
   DatetimeIndex

[period]
start_date = "2005-01-01" # ISO 8601 date, inclusive
end_date   = "2020-12-31" # ISO 8601 date, inclusive
```

Listado 3.1. Configuración de ubicación y periodo en fichero `TOML`

La latitud y longitud se definen en grados decimales, con los valores positivos apuntando al norte y este respectivamente. También se determina la franja horaria (*timezone*), necesaria para los cálculos internos de PVGIS así como la indexación de las series temporales durante la fase de simulación.

Las fechas de inicio y fin (inclusivas) de la recogida de datos, y por lo tanto de la simulación, se definen como *YYYY-MM-DD*, es decir, *año-mes-día* en formato numérico.

Parámetros de ejecución (`[runtime]`).

Controlan el comportamiento del motor de simulación.

```
[runtime]
master_seed   = 42 # root entropy for per-panel-per-effect RNG
   seeding
num_panels    = 1000 # total fleet size
memory_cap_gb = 16.0 # maximum ram usage limit (aproximate) to be
   split across workers
num_workers   = 3 # 1 = single-core; >1 = multiprocessing.Pool size
```

Listado 3.2. Configuración de parámetros de ejecución (`runtime`)

El primer parámetro, `master_seed`, es la semilla raíz de la que se deriva, de forma reproducible, toda la aleatoriedad del sistema. Es decir, la misma cantidad de paneles ejecutada usando la misma semilla producirán invariablemente los mismos resultados.

El valor de `num_panels` indica la cantidad de paneles a simular.

Se establece el número de procesos paralelos con el parámetro `num_workers`, donde el valor 1 lanza una ejecución secuencial sobre un único proceso, mientras que valores superiores activan el procesamiento mediante el módulo `multiprocessing`.

Se establece un límite de memoria RAM aproximado en `memory_cap_gb`, que el simulador reparte entre los procesos generados. Junto a `num_workers`, limita el uso de la memoria RAM por el simulador. El límite de memoria se divide entre el número de trabajadores de forma equitativa, y se ajusta cada *chunk* de paneles al máximo número de paneles que caben en este espacio de memoria. Es decir, el tamaño de los fragmentos `parquet` escritos a disco vienen definidos por

$$parquet_chunk_size \leq \frac{memory_cap_gb}{num_workers}$$

En secciones posteriores se indaga más en profundidad en la forma de reparto de trabajo y escritura a memoria de los resultados de simulación.

Rutas (`[paths]`).

Especifican el fichero de entrada (`intake_file`, producto de la fase de adquisición u otra fuente de datos conforme al formato esperado por el simulador) y el directorio de salida (`output_dir`) donde se escriben los conjuntos de datos en formato Parquet.

```
[paths]
intake_file = "inputs/input_data.csv" # output of meteorology_helpers
   /gather_data.py -c
output_dir  = "output/" # chunked parquet files written here
```

Listado 3.3. Configuración de rutas

El archivo `intake_file` debe ser un CSV cuya primera columna `time` sea una fecha y hora con zona horaria (por ejemplo *2005-01-01 00:00:00+00:00*), que se emplea como índice de la serie. Además, debe incluir los siguientes campos:

- `poa_direct`: irradiancia directa proyectada sobre el plano del array (*Plane of Array*, POA), en W/m^2.
- `poa_sky_diffuse`: componente difusa de cielo de la irradiancia POA, en W/m^2.
- `poa_ground_diffuse`: componente difusa reflejada por el suelo de la irradiancia POA, en W/m^2.
- `poa_global`: irradiancia global incidente sobre el plano, suma de las tres componentes POA, en W/m^2.
- `temp_air`: temperatura ambiente del aire, en grados Celsius.
- `wind_speed`: velocidad del viento, en m/s.
- `solar_zenith`: ángulo cenital solar aparente, en grados.
- `solar_azimuth`: ángulo acimutal solar, en grados.
- `otras columnas`: cualquier dato requerido por los modelos implementados de efectos o fallos sobre los módulos fotovoltaicos, por ejemplo, la humedad relativa.

Modelos físicos([singlediode],[temperature],[incident_angle]).

Seleccionan, mediante identificadores que se corresponden con `enums` del código el algoritmo concreto que PVLib empleará en cada etapa.

```
[singlediode]
method = "NEWTON" # one of SingleDiodeMethod enum members

[temperature]
model = "SAPM_OPEN_RACK_GLASS" # one of TemperatureModel enum members

[incident_angle]
model = "PHYSICAL" # one of IncidentAngleModel enum values
# Per-model glass parameters; only the set matching `model` is
   consumed.
n = 1.526 # PHYSICAL: refractive index
K = 4.0 # PHYSICAL: glazing extinction coefficient (1/m)
L = 0.002 # PHYSICAL: glazing thickness (m)
# b   = 0.05 # ASHRAE
# a_r = 0.16 # MARTIN_RUIZ
```

Listado 3.4. Configuración de modelos físicos

El método de resolución del modelo de diodo único puede tomar los valores `NEWTON`, `LAMBERTW`, `BRENTQ` o `CHANDRUPATLA`. Debido a limitaciones de hardware y restricciones de tiempo, se recomienda encarecidamente el uso del método `NEWTON`, sobretodo para grandes volúmenes de datos.

El modelo térmico de cálculo de la *cell temperature* permite las variantes SAPM y PVsyst.

El modelo de modificador por ángulo de incidencia o IAM puede ser `PHYSICAL`, `ASHRAE` o `MARTIN_RUIZ`). La sección `[incident_angle]` incluye además los parámetros de cada opción, y consume únicamente el subconjunto que corresponde al modelo seleccionado.

Módulo y montaje ([module], [mounting]).

Define los parámetros de los módulos fotovoltaicos simulados, así como su orientación. En esta primera versión de la herramienta se asume una orientación y modelo fotovoltaico uniformes.

```
[module]
# CEC single-diode reference parameters, taken verbatim from the CEC
   module
# database entry for the Canadian Solar CS3W-400P (400 W, 72-cell
   multi-Si),
# a widely deployed utility/commercial module.
alpha_sc = 0.002409
a_ref    = 1.756127
I_L_ref  = 10.904441
I_o_ref  = 2.303482e-11
R_sh_ref = 741.889771
R_s      = 0.302266
Adjust   = 3.759108

[mounting]
# Array orientation
surface_tilt    = 30.0 # degrees from horizontal
surface_azimuth = 180.0 # degrees clockwise from north
```

Listado 3.5. Configuración de módulo CEC y montaje físico

La sección [module] contiene los siete parámetros de referencia del modelo de diodo único en la parametrización CEC, tomados de la correspondiente entrada de la base de datos.

La sección [mounting] fija la orientación de la instalación mediante la inclinación y el acimut de la superficie, asumiendo una montura fija. Las monturas dinámicas como rastreador de eje único (*single-axis tracker*) quedan fuera del alcance de esta primera iteración.

Efectos (`[[effects.*]]`).

Esta sección agrupa todos los efectos, modos de degradación y fallos que se incluirán en las subsecuentes simulaciones.

```toml
[[effects.soiling_variant]]
name                  = "soiling" # attribution column prefix; must
   be unique across effects
max_soiling_effect    = 0.3 # cap on the fractional irradiance loss
   (0..1); 0.3 = up to 30% loss
avg_hourly_progression = 1.0e-5 # mean per-hour increase in the
   soiling fraction (positive drift)
std_deviation         = 3.0e-5 # per-step noise on the progression;
   negative draws model cleaning
# Initial-soiling distribution (optional; fall back to the
   constructor defaults if omitted)
starting_median_fraction = 0.25 # median initial level as a fraction
   of max_soiling_effect
starting_shape_sigma     = 0.5 # sigma of the lognormal initial-level
    draw; larger = wider spread
```

Listado 3.6. Configuración de efectos, fallos y modos de degradación

Los efectos se declaran como arrays de tablas anidados, indexadas con una clave externa por cada clase concreta de efecto, que debe coincidir con el atributo `type` de la clase, y una entrada interna por cada instancia registrada de dicha clase. Las claves de cada entrada se pasan directamente como argumentos al constructor del efecto `type`.

El campo `name`, que debe ser único en todo el fichero, se emplea como prefijo de la columna de atribución correspondiente en la salida `parquet` de la simulación. Este diseño permite activar o desactivar cada efecto, e incluso instanciar variantes de un mismo efecto simultáneamente sin cambiar el código.

3.3.2. Validación de la configuración

Antes de iniciar la ejecución de la pipeline principal, la configuración se somete a un barrido de validación (ver la función `validate_config_file` en la documentación del código) que sigue una política de agregación de problemas. En lugar de abortar tras el primer error, recopila todas las violaciones detectadas y las notifica conjuntamente en una única excepción, de modo que el usuario pueda corregirlas todas de una vez.

Las comprobaciones cubren:

- La presencia de las configuraciones y elementos obligatorios.
- La coherencia de los valores de `[runtime]`.
- La validez de los identificadores de modelo frente a los `enums` presentes en código.
- La presencia de los parámetros de vidrio exigidos por el modelo IAM elegido.
- La existencia de los parámetros CEC.
- La correcta declaración de los bloques de efectos:
    - Que su tipo corresponda a una clase registrada.
    - Que se declaren como array de tablas.
    - Que no existan nombres de efecto duplicados.

El proceso de validación no comprueba la correctitud física de los valores, responsabilidad que recae sobre el usuario.

3.3.3. Ejecución de helpers meteorológicos

La adquisición de datos de los endpoints `PVGIS` y `Open-Meteo` se puede ejecutar mediante la herramienta `meteorology_helpers/gather_data.py`, que recibe la ruta del fichero de configuración y un directorio de salida.

Esta herramienta se puede lanzar desde la consola de comandos en tres modos de funcionamiento distintos usando *flags*:

- `--solpos`: Permite recuperar los datos suministrados por PVGIS. La salida se guarda en el archivo `solar_position.csv`.
- `--omet`: Solicita a `Open-Meteo` los campos definidos en el archivo `meteo.py`. La salida se guarda en `meteorology.csv`.
- `-c`: Descarga los datos de ambas fuentes y, además de los archivos anteriores, combina los resultados en el fichero `input_data.csv`, listo para el consumo de la pipeline general.

Las flags obligsatorias `--cfg` y `--output-dir` configuran la entrada y la salida de la herramienta auxiliar. En concreto, `--cfg` debe apuntar al archivo TOML de configuración del proyecto, mientras que `--output-dir` debe ser un directorio de salida válido que la herramienta pueda crear o escribir. El directorio puede existir previamente, o se creará en el momento de la ejecución. Si el directorio existe y contiene archivos con el mismo nombre que alguna de las salidas generadas, serán sobreescritos.

```
python meteorology_helpers/gather_data.py -c --cfg cfg.toml
   --output-dir ./inputs/
```

Listado 3.7. Comando de CLI para la adquisición de datos de entrada de SOLARIS

Al tratarse de un paso independiente, los datos se descargan una sola vez y pueden reutilizarse en múltiples ejecuciones de la simulación.

3.3.4. Ejecución de simulación

La simulación se lanza mediante `main.py`, indicando con `--cfg` la ruta del fichero de configuración. Tras la validación descrita, SOLARIS coordina la ejecución completa. Toda la configuración se relega al archivo de configuración.

```
python main.py --cfg cfg.toml
```

Listado 3.8. Comando de CLI para la ejecución de SOLARIS

3.4. Simulación

La siguiente sección sirve como una descripción general del proceso completo de simulación llevado a cabo por SOLARIS.

No se debe entender como un sustituto a la documentación del proyecto, sino como un *primer* para entender, a alto nivel, la estructura subyacente y funcionamiento interno.

3.4.1. Flujo de ejecución general

El flujo general del programa se puede entender como una serie de pasos secuenciales, tal y como se observa en la figura 3.1.

1. **Configuración:** Se lee el fichero `.toml` de entrada y se procesan los datos contenidos.

   Se preparan los datos de condiciones de entorno (`env_params`), que aúna irradiancias, variables meteorológicas y otras variables relevantes para distintos efectos; y el marco de posiciones solares (`solar_positions`), empleado en el cálculo del ángulo de incidencia.

   Se resuelven los enumerados que definen los modelos de `pvlib` a usar durante la simulación, y se cargan los parámetros del modelo fotovoltaico y las variables de los efectos, fallos y degradaciones implementados.

2. **Construcción de parámetros y efectos:** Se instancian los objetos de efecto a partir de los bloques `[[effects.*]]`. Cada efecto recibe sus variables del archivo de configuración e instancian una clase que permite precalcular el punto de comienzo y la progresión de cada pareja panel-efecto.

   El módulo `effect_generator` se configura con la lista de efectos, la semilla maestra y la longitud del eje temporal, y se valida que cada efecto declare como afectados únicamente los parámetros válidos definidos en SOLARIS, que son los cuatro de los cinco parámetros CEC, la irradiancia y la temperatura de las *cells*.

3. **Preparación del directorio de salida.** Se crean los subdirectorios `observables/` y `attribution/` en el *path* configurado y se exige que estén vacíos. El motor se niega a escribir en un directorio no vacío para impedir que una nueva ejecución mezcle de forma silenciosa sus ficheros con los de una ejecución anterior, independientemente del tipo de archivos que se encuentren en los directorios.

4. **Fragmentación de la carga.** La flota se divide en fragmentos (*chunks*) cuyo tamaño se calcula a partir del límite de memoria configurado. A cada proceso de trabajo se le asigna la fracción

$$\frac{memory_cap_gb}{num_workers}$$

del total, que acota cuántos paneles caben simultáneamente en memoria. El caso ideal, dado que el problema es *embarrassingly parallel* y limitado por CPU, es un fragmento por proceso. Si la división equitativa no cabe en el máximo de memoria, se incrementa el número de fragmentos hasta que quepa, procesándose entonces en oleadas de *chunks* que finalizan aproximadamente a la vez (suponiendo un hardware idéntico en cada núcleo).

5. **Ejecución del bucle por panel.** Si `num_workers` es 1, los fragmentos se procesan secuencialmente; en caso contrario se distribuyen entre un grupo (*pool*) de procesos. Dado que los procesos lanzados con `spawn` reimportan los módulos con el estado vacío, un inicializador reconfigura en cada proceso el generador de efectos y comparte las entradas de simulación. El avance se refleja en una barra de progreso (`tqdm`) sobre el total de paneles, además de `logs` que ofrecen información acerca de la ejecución, por ejemplo, los efectos incluidos.

6. **Escritura de resultados.** Cada fragmento, una vez simulado, se serializa de inmediato como un fichero parcial (*part-file*) parquet dentro de los conjuntos de datos observables y de atribución.

3.4.2. Abstracción de Modelos de Efecto, Fallo, y Degradación

El conjunto de fenómenos que pueden alterar la producción de un módulo es largo y variado, desde la acumulación gradual de suciedad hasta fallos de origen eléctrico o brechas en el propio módulo. SOLARIS los unifica bajo un

único contrato abstracto, con el objetivo de que el núcleo de la simulación no dependa de las particularidades de cada uno. El generador de efectos y el generador de datos operan siempre contra esa interfaz, de modo que incorporar un fenómeno nuevo no requiere modificar el flujo principal, sino únicamente proporcionar una implementación acorde al contrato.

Interfaz Abstracta

Todo modelo de efecto hereda de la clase abstracta `AbstractBaselineFailure`, que declara la interfaz mínima. Cada subclase encapsula su distribución de aparición, su propia matemática de aceleración, y su dinámica de progresión; y expone dos atributos de clase y dos métodos:

- `type`: Atributo identificador que `main.py` emplea para localizar los parámetros del efecto en el fichero de configuración, bajo la tabla anidada `[[effects.<type>]]`.
- `affected_columns`: Atributo tipo tupla que recoge los parámetros característicos que el modelo modifica. Declararlos por adelantado permite validar que solo se modifican los parámetros soportados por SOLARIS, y que no se ha introducido ningún error o errata a la hora de implementar el modelo.
- `compute_time_to_onset(seed)`: Determina el primer instante en que el efecto se activa para un panel. Un valor igual o superior a la longitud de la serie significa que el efecto no llega a manifestarse.
- `compute_progression(seed, start, end)`: devuelve la curva de progresión del efecto desde su aparición hasta el final del horizonte, como un `DataFrame` indexado por tiempo cuyas columnas son un subconjunto de los ejes válidos.

Aplicación del efecto

Un efecto se manifiesta modificando el conjunto cerrado de seis parámetros, cada cual correspondiente a una entrada necesaria para el cálculo de la curva I-V. Un efecto no manipula la curva directamente, sino se limita a declarar

sobre qué parámetros actúa y a producir, para cada instante, el valor de modificación correspondiente.

Eje	Magnitud afectada	Composición	Valor
`G`	Irradiancia efectiva	Multiplicativa	fracción $[0, 1]$
`I_L`	Fotocorriente	Multiplicativa	fracción $[0, 1]$
`R_sh`	Resistencia *shunt*	Multiplicativa	fracción $[0, 1]$
`T`	Temperatura de célula	Aditiva	°C
`R_s`	Resistencia serie	Aditiva	Ω
`I_0`	Corriente de saturación	Aditiva	A

Tabla 3.1. Parámetros susceptibles a efectos y reglas de composición.

`nNsVth` se omite de los modificadores válidos ya que se trata de un parámetro derivado, y no una medida estrictamente física.

La regla de composición depende de la naturaleza física de la magnitud.

Los parámetros **multiplicativos** representan pérdidas fraccionarias $r \in [0, 1]$ (un $r = 0{,}3$ significa un 30 % de pérdida en este instante) y el consumidor las aplica como $baseline \cdot (1 - r)$.

Los parámetros **aditivos** representan incrementos físicos en la unidad del parámetro y el consumidor los aplica como $baseline + \delta$.

Dado que varios efectos pueden manifestarse a la vez en un mismo módulo, el motor de composición combina, para cada panel, todos los efectos registrados en un único conjunto de modificadores. Recorre los efectos, deriva para cada uno su semilla independiente, determina su instante de aparición y, si aplica, obtiene su progresión y la acumula sobre un vector por parámetro según la regla correspondiente. Cuando varios efectos actúan sobre la misma entrada, sus contribuciones se combinan:

$$\text{multiplicativa: } \prod_i (1 - r_i) \qquad \text{aditiva: } \sum_i \delta_i$$

Esta separación es la clave de la abstracción. Los efectos son agnósticos entre sí y respecto a sus reglas de agregación. Cada uno declara sus parámetros afectados y produce su progresión de forma aislada, y el motor de composi-

ción resuelve los solapamientos. El resultado son dos estructuras por panel: los modificadores compuestos, que el generador de datos aplica sobre los parámetros, y los instantes de aparición de cada efecto, que alimentan el conjunto de atribución.

La independencia estadística entre paneles y entre efectos se garantiza derivando cada semilla a partir de una semilla maestra, empleando el par `(panel, efecto)` como clave. Dos ejecuciones con la misma semilla maestra producen resultados idénticos, y los flujos aleatorios de paneles y efectos distintos son mutuamente independientes.

Registro, configuración y validación

Los modelos disponibles se enumeran en un registro central en el archivo `__init__.py` del directorio `failure_types`, donde deben residir las implementaciones de los modelos de efecto. Al cargar la configuración, `main.py` recorre dicho registro y, para cada clase, busca sus entradas `[[effects.<type>]]` en el fichero TOML, instanciando un objeto por entrada y pasando las claves de la tabla como argumentos del constructor. Incorporar un fenómeno nuevo se reduce, por tanto, a tres pasos que no modifican el código *core* de SOLARIS:

1. Escribir una subclase que implemente la interfaz heredando la clase abstracta.
2. Añadir la nueva clase al registro en `__init__.py`.
3. Declarar el (o las variantes del) efecto y sus parámetros en el archivo de configuración.

Antes de iniciar la simulación, se realiza una validación para comprobar que cada efecto declara al menos un parámetro afectado, que todos los parámetros declarados pertenecen al conjunto válido, y que no existen nombres de efecto duplicados. Como el camino de composición en tiempo de ejecución no realiza verificaciones adicionales, esta comprobación de arranque es la única validación frente a declaraciones erróneas, y reúne todos los incumplimientos en un único mensaje para que el usuario los vea y solucione a la vez.

3.4.3. Generador de efectos

El generador de efectos es el motor que traduce el conjunto de modelos configurados en modificadores concretos para cada panel. Consume las implementaciones que respetan la interfaz abstracta descrita anteriormente y entrega al generador de datos los modificadores que este aplicará sobre el modelo de simulación. De este modo, ni los modelos de efecto conocen cómo se combinan entre sí, ni el generador de datos conoce las matemáticas internas de cada fenómeno, todo queda encapsulado en el generador de efectos.

Configuración y estado idéntico

El generador mantiene su configuración en estado de módulo, el conjunto de efectos registrados, la semilla maestra y la longitud de la serie temporal se fijan una sola vez al arrancar, mediante una llamada de configuración previa al bucle por panel.

Esta decisión evita tener que arrastrar dichos parámetros por toda la cadena de llamadas, pero introduce una consideración bajo ejecución multiproceso. Los trabajadores se crean con el método `spawn`, generando procesos nuevos, con lo que cada uno reimporta el módulo con el estado vacío. Por ello, la configuración debe reaplicarse dentro de cada uno a través de su inicializador, de forma que todos partan de un estado idéntico al del proceso principal antes de procesar su primer *chunk*.

Salida

El generador devuelve dos estructuras. La primera es un diccionario de modificadores que contiene los parámetros afectados por algún efecto. El generador de datos posteriormente consume estos modificadores para perturbar los parámetros de entrada del modelo *SDM*.

La segunda estructura es un diccionario de instantes de aparición, que asocia cada efecto activado con el paso en que se manifestó. De él se genera el conjunto de atribución, que registra la trazabilidad de qué le ocurrió a cada módulo y cuándo.

3.4.4. Generador de datos

El generador de datos es el motor de simulación propiamente dicho. Recibe un *chunk* de paneles junto con las variables de entorno y produce los dos conjuntos de resultados, observables y atribución. Es la unidad de trabajo que invocan los trabajadores descritos en la ejecución multiproceso.

Preparación por chunk

Buena parte del cálculo no depende de la identidad del panel, sino únicamente del entorno y de los parámetros del módulo, comunes a todo el *chunk*. Estas magnitudes se calculan una sola vez, antes del bucle por panel:

- La **irradiancia efectiva** que alcanza las células, obtenida a partir de las componentes de irradiancia en el plano y de los modificadores de ángulo de incidencia (*IAM*) para las componentes directa y difusa, según el modelo IAM configurado.
- La **temperatura de célula**, estimada a partir de las condiciones ambientales mediante el modelo térmico seleccionado.

Ambas constituyen la línea base sobre la que actuarán los efectos. La única variación entre paneles proviene de los modificadores sobre los resultados finales, por lo que calcular la línea base una vez y aplicar después modificadores por panel es sustancialmente más eficiente que rehacer todo el cálculo en cada iteración.

En esta misma fase se reservan las estructuras de salida. El conjunto de observables se preasigna completo, indexado por `(panel_id, timestamp)` e inicializado a cero, y el de atribución con un valor nulo por celda. Preasignar permite escribir los resultados de cada panel en su bloque contiguo de filas, sin una concatenación final que duplicaría la memoria. Finalmente se calcula la máscara de filas iluminadas, compartida por todos los paneles del *chunk*. Este cálculo de máscara se detalla más adelante en la sección "3.4.5 Técnicas de optimización".

Bucle por panel

Para cada panel, se obtiene del generador de efectos sus modificadores compuestos y los instantes de aparición de cada efecto, y se ejecuta la cadena de simulado *CEC*. Los modificadores no se aplican en un único punto, sino en dos etapas que respetan la naturaleza de cada magnitud:

1. **Sobre las entradas ambientales.** Los modificadores de irradiancia (`G`) y de temperatura (`T`) se aplican sobre la línea base.
2. **Cálculo de parámetros de referencia.** A partir de la irradiancia y la temperatura modificadas y de los parámetros de referencia CEC del módulo, se obtienen los cinco parámetros del modelo *SDM*.
3. **Sobre los parámetros del diodo.** Los modificadores restantes (`I_L`, `I_0`, `R_s`, `R_sh`) se aplican sobre esos cinco parámetros recién calculados, cada uno según su regla de composición.
4. **Resolución de la curva.** Con los parámetros finales se resuelve el modelo de un diodo, obteniendo los siete puntos característicos de la curva I-V.

Esta separación en dos etapas es deliberada, ya que un efecto sobre la irradiancia debe propagarse a través del cálculo de parámetros, mientras que un efecto sobre los 5 parámetros *CEC* actúa directamente. La razón para recalcular los modificadores de irradiancia y temperatura de células por cada panel radica en efectos como brechas en el cristal, que pueden alterar estos valores y que influencian el resultado de los otros 5 parámetros.

3.4.5. Técnicas de optimización

Dada la carga de cómputo que acarrea la simulación de módulos fotovoltaicos con los posibles efectos y fallos que pueden sufrir, SOLARIS hace uso de principalmente dos técnicas de optimización para la aceleración de los cálculos.

Ejecución multiproceso

Como se viene mencionando en el resto del documento, la paralelización juega un papel clave en la ejecución del proyecto.

Dada la naturaleza *embarrassingly parallel* del problema, se ha optado por una división de trabajo sencilla que facilite el procesamiento paralelo de las operaciones así como las subsecuentes lecturas de los datos generados.

Puesto que existe un límite máximo de memoria impuesto por la configuración del usuario, esta se divide equitativamente entre el número de trabajadores paralelos configurados. Esto es:

$$mem_per_process = \frac{memory_cap_gb}{num_processes}$$

Este límite de memoria por proceso tiene capacidad para procesar un número determinado de los módulos a simular. Calcular este número conlleva medir de forma empírica el espacio que ocupa cada *timestep* de un panel en memoria, que resulta en $62B$. Se aproximan a $64B$ para asegurar un margen. Es decir, se asume que cada panel ocupa en memoria $64B$ por cada fila (*timestep*) de la serie temporal:

$$bytes_per_panel = num_timesteps \cdot 64B$$

Con esta medida en mente, se obtiene el número de paneles que un trabajador puede procesar a la vez:

$$max_panels_per_process = max(1, \frac{mem_per_process \cdot 10^9 B}{bytes_per_panel})$$

Dado que la forma más eficiente de procesar todos los módulos es repartir aproximadamente el mismo número de ellos a cada trabajador (para evitar sobrecargar o desaprovechar alguno de los procesos) se fuerza un reparto equitativo, ajustando el número de paneles por *chunk* para obtener la carga de trabajo más distribuida posible:

$$min_chunks = ceil(\frac{num_panels}{max_panels_per_process})$$

FInalmente, el número de *chunks* se escala al múltiplo más cercano del número de procesos disponibles, para evitar la problemática de uno o más procesos *idle* mientras se termina de procesar la última oleada:

$$chunks = num_processes \cdot ceil(\frac{min_chunks}{num_processes})$$

Máscara de filas nulas

Los módulos fotovoltaicos generan electricidad transformando la energía solar incidente. Los métodos utilizados para simular estos módulos operan los valores de entorno como irradiancia y temperatura de las células para estimar la salida de energía producida. Sin embargo, no poseen un filtro de irradiancia nula.

Todos los puntos característicos de la curva I-V son, en última instancia, un producto de la fotocorriente generada, que a su vez es proporcional a la irradiancia efectiva que experimenta el módulo. Ante una irradiancia nula el módulo no genera fotocorriente, por lo que todos sus puntos de la curva I-V son cero con independencia de los efectos o fallos aplicados. Resolver estas filas consume ciclos de CPU que no aportan ninguna información.

Para evitar esto, SOLARIS aplica una máscara a las filas nulas, eliminándolas de la serie temporal antes de realizar los cálculos necesarios para simular el panel. Esta máscara se calcula una sola vez y se reutiliza para todos los paneles. Puesto que los modificadores de irradiancia introducidos son multiplicativos, una fila que es nula en el caso base permanece nula para todos los paneles y efectos. Se puede, por tanto, excluir del cómputo con la garantía de que su resultado no cambiará.

Una vez resueltas las filas iluminadas, se reconstruye la serie temporal completa. Los resultados se reinsertan en sus posiciones originales aplicando la misma máscara, de modo que cada valor calculado regresa a su *timestep* correspondiente, y las filas oscuras se rellenan con ceros.

De esta forma se reduce el número de operaciones por panel en aproximadamente la mitad, ya que a lo largo de un año la fracción de horas en oscuridad ronda el 50 % de la serie temporal.

3.5. Formato de Resultados

Una vez completada la simulación, SOLARIS escribe sus resultados bajo el directorio configurado. La salida se organiza en dos conjuntos de datos independientes, cada uno en su propio subdirectorio:

- `observables`: Recoge la curva característica de cada panel en cada *timestep*.
- `attribution`: Registra el momento de aparición de cada efecto sobre cada panel.

Ambos conjuntos se almacenan en formato Parquet. Cada subdirectorio constituye un *dataset* de esquema único, de modo que un consumidor puede leerlo entero con una sola llamada (`read_parquet(dir)`) sin preocuparse de los ficheros individuales que lo componen.

```
output_dir/
|-- observables/
|   |-- observables_00000000_00000250.parquet
|   |-- observables_00000250_00000500.parquet
|   `-- ...
`-- attribution/
|   |-- attribution_00000000_00000250.parquet
|   |-- attribution_00000250_00000500.parquet
|   `-- ...
```

Cada *chunk* de la simulación produce un fichero (*part-file*) en cada subdirectorio. Su nombre incorpora el rango global de paneles que contiene, con el formato `<inicio>_<fin>` en ocho dígitos con relleno de ceros, siendo el límite superior exclusivo. Así, los ficheros quedan ordenados y no colisionan entre sí. SOLARIS exige que ambos subdirectorios estén vacíos antes de comenzar: mezclar ficheros de dos ejecuciones distintas en un mismo *dataset* produciría lecturas inconsistentes.

3.5.1. Conjunto de observables

El conjunto *observables* contiene los siete puntos primarios de la curva I-V resueltos para cada panel en cada *timestep*. Se indexa por el par `(panel_id, timestamp)`, con $num_panels \cdot num_timesteps$ filas en total. Cada una de las siete columnas es un valor de tipo `float64`:

Columna	Magnitud	Unidad
`i_sc`	Corriente de cortocircuito	A
`v_oc`	Tensión de circuito abierto	V
`i_mp`	Corriente en el punto de máxima potencia	A
`v_mp`	Tensión en el punto de máxima potencia	V
`p_mp`	Potencia máxima	W
`i_x`	Corriente en $V = V_{oc}/2$	A
`i_xx`	Corriente en $V = (V_{oc} + V_{mp})/2$	A

Tabla 3.2. Columnas del conjunto de observables.

Las columnas `i_x` e `i_xx` son dos puntos de muestreo adicionales definidos por el modelo SAPM que ayudan a reconstruir la forma de la curva con mayor resolución y fidelidad.

3.5.2. Conjunto de atribución

El conjunto *attribution* es un registro compacto de la trazabilidad de los efectos que se compone de una fila por panel, indexada por `panel_id`, y una columna por cada efecto registrado, nombrada `<efecto>_onset`. Cada celda almacena la marca temporal (`datetime`) en la que ese efecto se activó sobre ese panel, o un valor nulo (`NaT`) si el efecto no llegó a manifestarse dentro del tiempo simulado. Al haber una única fila por panel, este conjunto es mucho más pequeño que el de observables y actúa como índice de qué le ocurrió a cada módulo y cuándo.

4. Demostración

4.1. Testing

La reproducibilidad es una garantía central de SOLARIS. Dos ejecuciones con la misma semilla maestra deben producir resultados idénticos, y los flujos aleatorios de paneles y efectos distintos deben ser mutuamente independientes. La batería de pruebas se concentra en el motor de números aleatorios del generador de efectos, que es donde reside esa garantía. Para aislar el mecanismo de las particularidades de cualquier modelo concreto, las pruebas emplean un efecto sintético mínimo, cuyo instante de aparición y progresión se derivan únicamente de la semilla recibida. Así, se ejecuta el proceso de composición sin depender de la física de un fallo real.

Los tests implementados comprueban:

- **Determinismo**: una misma tupla produce siempre la misma semilla, independientemente del número de llamadas realizadas.
- **Separación de panel y efecto**: variar el panel o el efecto cambia la semilla resultante, así como el orden de ambas ya que la combinación no es conmutativa, de modo que pares distintos nunca se confunden entre sí.
- **Dependencia de la semilla maestra**: cambiar la semilla maestra cambia la semilla derivada para un mismo par `(panel, efecto)`.
- **Reproducibilidad**: recalcular los modificadores de un mismo panel devuelve instantes de aparición y vectores de modificación idénticos bit a bit.
- **Independencia entre paneles**: paneles distintos poseen flujos independientes, por lo que los instantes de aparición de efectos no son todos iguales.

- **Independencia del particionado**: procesar un mismo conjunto de identificadores globales bajo dos particionados en *chunks* distintos verifica que cada panel recibe modificadores idénticos en ambos casos.

4.2. Implementación de un efecto

Se ha implementado, a modo de ejemplo así como validación de la arquitectura y el sistema SOLARIS, un efecto sencillo que demuestra el procedimiento para implementar otros efectos a través de la interfaz abstracta habilitada.

El efecto implementado es un modelo sencillo de acumulación de suciedad en el módulo, que como la mayoría de modelos de degradación siempre aparece en el primer *timestamp* y con un estado inicial de suciedad que sigue una distribución *lognormal*. Posteriormente se sigue un *random walk* con *drift* positivo, es decir, en función de los parámetros de configuración, cada panel se acercará más o menos rápido a un máximo de suciedad acumulada establecido. Esta suciedad acumulada se manifiesta en los parámetros *CEC* como una reducción fraccional en la irradiancia (`G`).

Los parámetros de configuración leídos del archivo TOML son los siguientes, y permiten configurar diversos aspectos del efecto:

- `max_soiling_effect`: cota asintótica de la pérdida de irradiancia, expresada como fracción en $[0, 1]$. Por ejemplo, un valor de $0{,}3$ limita la pérdida acumulada a un máximo del 30 %. Es el techo que el *random walk* nunca supera.
- `avg_hourly_progression`: incremento medio por paso de la fracción de suciedad, es decir, el *drift* positivo del *random walk*. Gobierna la rapidez con que un panel se aproxima al máximo ya que cuanto mayor sea su valor, más rápido se ensucia el panel.
- `std_deviation`: desviación típica del incremento en cada paso. Introduce la aleatoriedad del recorrido y, mediante sus valores negativos, modela episodios de ligera limpieza parcial por lluvia o viento.
- `starting_median_fraction` (opcional): mediana del nivel inicial de suciedad, expresada como fracción de `max_soiling_effect`. Determina

el punto de partida típico del conjunto de paneles. Por defecto, $0{,}25$.

- `starting_shape_sigma` (opcional): desviación típica de la distribución normal subyacente a la *lognormal* del nivel inicial. Valores mayores ensanchan la dispersión de los estados iniciales entre paneles, produciendo mayor diversidad en el punto de partida. Por defecto, $0{,}5$.
- `name`: identificador único del efecto, empleado como prefijo de su columna en el conjunto de atribución.

Las figuras 4.1 y 4.2 agregan la producción simulada de 10.000 paneles a lo largo de 16 años, donde se puede observar el efecto del modelo de acumulación de suciedad, reduciendo la potencia generada.

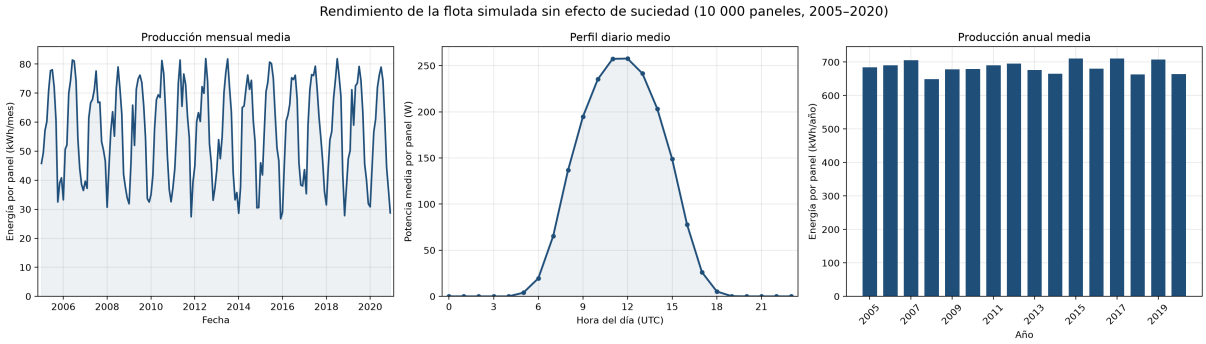


Figura 4.1. Producción energética por módulos fotovoltaicos sin acumulación de suciedad.

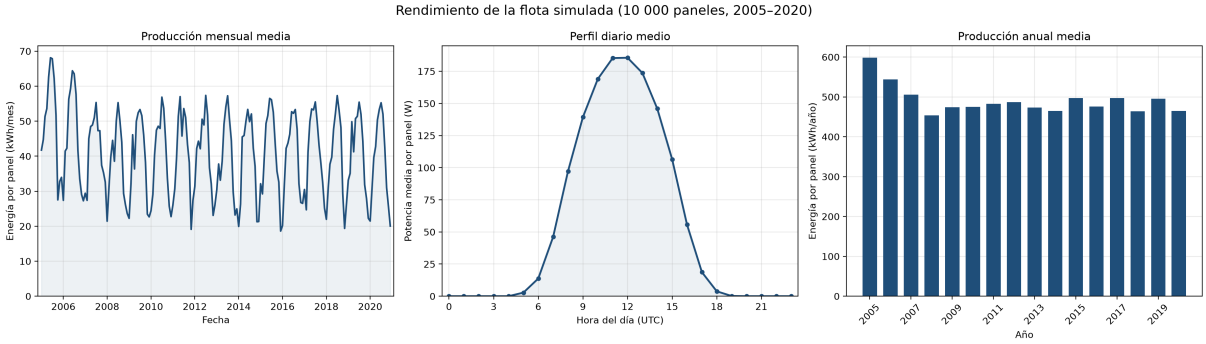


Figura 4.2. Producción energética por módulos fotovoltaicos con acumulación de suciedad.

Se puede apreciar una clara reducción en la potencia generada por los módulos fotovoltaicos que sí modelan la acumulación de suciedad durante los

primeros años, estabilizándose una vez se alcanza el límite establecido por la configuración.

La arquitectura de SOLARIS se centra en la composición de efectos. La figura 4.3 representa un caso de composición que emplea dos instancias del modelo de acumulación de suciedad simultáneamente, configurados de forma que se alcance un *soft limit* del 30 % similar al que se observa en la ejecución con 1 solo efecto, y que posteriormente continúa progresando mucho más lentamente.

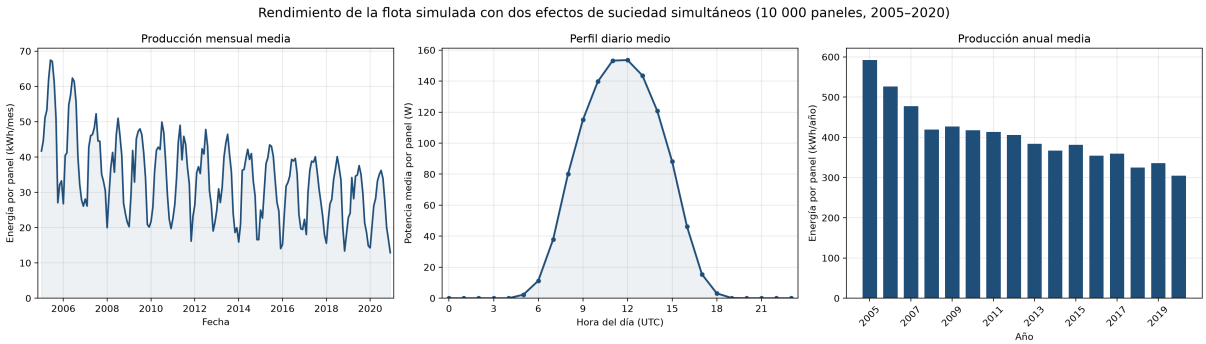


Figura 4.3. Producción energética por módulos fotovoltaicos con acumulación de suciedad en dos fases.

4.3. Análisis de rendimiento

Esta sección analiza la escalabilidad de SOLARIS respecto al número de procesos configurados, con el objetivo de evaluar en qué medida la paralelización (Sección 3.4.5) consigue una mejora de rendimiento. Para ello se ejecutó una simulación idéntica de 10.000 paneles a lo largo de 16 años (140.256 pasos temporales horarios), con un único efecto activo (*soiling*), variando el número de procesos de trabajo entre 1 y 4, sobre el mismo hardware de consumidor (MacBook Air M4).

Procesos	Tiempo total (s)	Rendimiento (paneles/s)	Aceleración	Eficiencia
1	1194.0	8.37	1.00×	100.0 %
2	713.0	14.03	1.67×	83.7 %
3	600.0	16.67	1.99×	66.3 %
4	555.1	18.02	2.15×	53.8 %

Tabla 4.1. Tiempo de ejecución, rendimiento, aceleración y eficiencia de la simulación (10000 paneles, 16 años, un único efecto) en función del número de procesos de trabajo.

La aceleración obtenida es notablemente inferior a la ideal, y la brecha se ensancha de forma sostenida a medida que se añaden procesos. Con 4 procesos se alcanza una aceleración de apenas 2,15× frente al 4× teórico, una eficiencia del 53.8 %. El rendimiento por proceso individual decrece de forma monótona en las cuatro configuraciones, confirmando que el problema, aunque *embarrassingly parallel* en su formulación teórica (Sección 3.4.5), está lejos de escalar linealmente en la práctica.

Ajustando los cuatro puntos observados al modelo de la ley de Amdahl, la fracción de trabajo secuencial estimada a partir de cada configuración es de 19.4 % ($n = 2$), 25.4 % ($n = 3$) y 28.7 % ($n = 4$). A diferencia de lo que predice un modelo de Amdahl clásico, donde esta fracción debería ser constante e independiente de n, dado que corresponde a un cuello de botella fijo del sistema (inicialización del `Pool`, reparto de *chunks*, escritura final de resultados...), la estimación aquí crece con el número de procesos empleados. Este patrón descarta un cuello de botella único y fijo como explicación completa, y apunta en su lugar a una fuente de sobrecarga cuyo coste aumenta con el grado de paralelismo.

Esta interpretación se ve reforzada por una observación cualitativa directa. Durante las distintas ejecuciones se apreció un calentamiento notable del equipo a medida que avanzaba la simulación, coherente con el *throttling* térmico esperable en un MacBook Air M4, que carece de ventilación activa. La propia progresión de los tiempos por *chunk* respalda esta hipótesis. En el caso de 4 procesos, los primeros *chunks* completados se resuelven en torno a 81 s, mientras que los tiempos alcanzan un máximo de 99.87 s hacia el final de la ejecución. Un incremento superior al 20 % entre el *chunk* más rápido

(78.32 s) y el más lento constituye un patrón de degradación progresiva.

Este patrón se reprodujo en ejecuciones posteriores e independientes sobre el mismo hardware. En una ejecución con 4 procesos y sin ningún efecto activo, los *chunks* de la primera oleada se completaron en torno a 75 s, mientras que los de las últimas oleadas alcanzaron los 94.63 s, una degradación cercana al 27 % con una carga por panel estrictamente menor. Una segunda ejecución con dos efectos activos mostró el mismo patrón (de 81.5 s en la primera oleada a un máximo de 104.96 s al final). La aparición sistemática en tres ejecuciones con cargas de trabajo distintas, siempre partiendo de tiempos iniciales bajos y degradándose de forma monótona con el avance de la simulación, es consistente con un fenómeno dependiente del estado térmico de la máquina y no de la carga computacional concreta, lo que refuerza el *throttling* como causa principal de la sobrecarga creciente observada.

Junto al *throttling* térmico, se suma la contribución de otras fuentes de sobrecarga creciente con n, como la E/S en la escritura simultánea de resultados, el coste de serialización (*pickle*) de los datos repartidos y recolectados por el `Pool`, y la contención por ancho de banda de memoria y caché de último nivel entre los núcleos activos, cuya presión aumenta con el número de procesos ejecutándose de forma simultánea.

Aislar la contribución relativa de cada una de estas causas repitiendo el experimento en hardware más potente, en un entorno software aislado, o instrumentando la ejecución para distinguir tiempo de cómputo de tiempo de E/S y comunicación, excede el alcance de este PFG y se plantea como línea de trabajo futuro.

Cabe destacar que el modelo de paralelización basado en múltiples procesos independientes impone un coste de serialización y transmisión de datos entre el proceso principal y cada trabajador. Una vía de trabajo futuro prometedora pasa por una reimplementación de SOLARIS en un lenguaje con soporte de *multithreading* real sobre memoria compartida (por ejemplo, Rust o C++), donde múltiples hilos podrían operar directamente sobre los mismos datos sin necesidad de serialización ni copia entre procesos.

Esta reimplementación permitiría además implementar estrategias de paralelización más avanzadas, como puede ser el micro-*chunking* dinámico, donde los *chunks* reducen en gran medida su tamaño (alrededor de 50-100 pa-

neles por chunk) y con ello reducir posibles fallos de memoria caché que ralentizarían la ejecución.

En conjunto, los resultados obtenidos permiten concluir que la paralelización implementada en SOLARIS ofrece una mejora de rendimiento clara, reproducible y monótonamente creciente con el número de procesos, pero con una eficiencia marginal decreciente cuyo origen más probable es una combinación de limitaciones térmicas del hardware de prueba y sobrecarga entre procesos, más que una limitación estructural del esquema de paralelización elegido.

4.3.1. Coste computacional de los efectos

Además de la escalabilidad respecto al número de procesos, resulta relevante caracterizar el coste que introduce la propia inyección de efectos, dado que constituye la funcionalidad diferencial de SOLARIS frente a un simulador convencional. Para ello se ejecutó la misma simulación de referencia (10.000 paneles, 16 años, 4 procesos) en tres configuraciones: sin ningún efecto activo, con una instancia del modelo de suciedad, y con dos instancias del mismo modelo actuando de forma compuesta.

Efectos activos	Tiempo total (s)	Rendimiento (paneles/s)	Coste
0	494.6	20.22	—
1	555.1	18.02	+60.5 s (+12.2 %)
2	563.3	17.75	+68.7 s (+13.9 %)

Tabla 4.2. Tiempo de ejecución y rendimiento de la simulación en función del número de efectos activos.

El primer efecto introduce un coste del 12.2 % respecto a la línea base, mientras que el segundo añade únicamente un 1.5 % adicional. Este escalado marcadamente sublineal es consecuencia directa de la arquitectura de composición descrita en la Sección 3.4.3. Dado que ambas instancias actúan sobre el mismo parámetro (la irradiancia efectiva), el motor de composición las agrega en un único vector de modificadores multiplicativos, y la resolución del

modelo de diodo único, que domina el coste computacional por panel, se ejecuta una sola vez con independencia del número de efectos activos. El coste marginal de cada efecto adicional se reduce así al cálculo de su instante de aparición y su curva de progresión.

Debe señalarse que estas cifras proceden de una única ejecución por configuración, sobre un hardware cuya deriva térmica introduce una variabilidad por ejecución superior a la diferencia observada entre las configuraciones de uno y dos efectos. Los resultados deben interpretarse por tanto como consistentes con un escalado sublineal del coste por efecto, y no como una medida precisa de este. Asimismo, el coste concreto depende de la complejidad computacional del modelo de efecto empleado. Los efectos con progresiones más costosas de calcular, o que actúen sobre parámetros distintos, presentarán costes diferentes.

5. Conclusiones

5.1. Análisis frente a los objetivos establecidos

A continuación se analiza el grado de cumplimiento de cada uno de los objetivos específicos planteados en la sección introductoria, a la luz del sistema finalmente implementado.

En conjunto, los seis objetivos específicos se consideran alcanzados con distinto grado de evidencia empírica. Los objetivos de generación de datos, modularidad, rendimiento y robustez cuentan con mecanismos concretos e implementados en el sistema, mientras que la extensibilidad y, en particular, la versatilidad se sostienen principalmente sobre el diseño del sistema, careciendo de una validación práctica exhaustiva dentro del alcance del PFG.

5.1.1. Generación de datos sintéticos a partir de principios físicos

Este objetivo se considera satisfecho. SOLARIS genera curvas I-V y series temporales de rendimiento a partir del modelo de diodo único, cuyos cinco parámetros físicos (I_L, I_0, R_s, R_{sh}, nN_sV_{th}) se derivan de la base de datos CEC mediante `calcparams_cec` y se perturban de forma trazable para modelar cada efecto. Se pueden simular 10.000 paneles solares a lo largo de 16 años en hardware de consumidor (MacBook Air M4), tomando alrededor de 10 minutos con un rendimiento de 16.67 paneles/s, repartiendo la carga de trabajo entre 3 procesos. Ha de tenerse en cuenta que solo se ha aplicado 1 efecto a esta prueba, y que la velocidad de simulación depende de la carga computacional subyacente en cada efecto.

5.1.2. Modularidad

Este objetivo se considera satisfecho. La configuración mediante el fichero TOML permite habilitar, deshabilitar y parametrizar de forma independiente cada efecto sin necesidad de modificar el código fuente. El barrido de validación (`validate_config_file`) garantiza además que cualquier combinación sea coherente antes de iniciar la simulación, lo cual refuerza la modularidad al permitir experimentar con configuraciones sin temor a fallos silenciosos a mitad de ejecución.

La interfaz abstracta para la implementación de efectos ofrece una forma limpia y mínimamente invasiva de extender las capacidades de simulación, añadiendo y excluyendo modelos según los requisitos del usuario.

5.1.3. Extensibilidad

Este objetivo se considera satisfecho, si bien es el más difícil de verificar de forma objetiva sin datos de uso a largo plazo. La arquitectura abstracta diseñada para el soporte de efectos donde cada nuevo modo de fallo se declara como una clase registrada frente al sistema de validación de configuración permite añadir nuevos modelos de degradación sin modificar el núcleo de la simulación.

5.1.4. Versatilidad

Este objetivo se considera parcialmente satisfecho dentro del alcance de este TFG. Si bien la arquitectura no impone restricciones que limiten su uso a la generación de datos de entrenamiento, usos alternativos como estudios de viabilidad o validación de algoritmos existentes no han sido explorados ni validados experimentalmente durante el desarrollo del proyecto, por lo que la versatilidad se sostiene como propiedad potencial más que como resultado demostrado.

5.1.5. Rendimiento, compatibilidad y escalabilidad

Este objetivo se considera satisfecho. La paralelización mediante múltiples procesos, explicada en la sección 3.4.5, permite un aprovechamiento efectivo de múltiples núcleos sin las limitaciones del `GIL` de Python. La dependencia exclusiva de bibliotecas multiplataforma (PVLib, pandas, NumPy, PyArrow) garantiza además la ejecución en cualquier sistema operativo compatible con Python.

Por ejemplificar, una ejecución secuencial monoproceso es capaz de simular 8.37 paneles/segundo, terminando un volumen de 10.000 paneles a lo largo de 16 años simulados en 1194 segundos.

Esa misma simulación, empleando 2 procesos reduce su tiempo de ejecución a 713 segundos, llegando a procesar en torno a 14 paneles por segundo.

Si se emplean 3 procesos, SOLARIS es capaz de procesar 16.67 paneles/segundo, completando la simulación en 600 segundos.

Con 4 procesos, el tiempo de ejecución desciende a 555 segundos, alcanzando un rendimiento de 18.02 paneles/segundo, una aceleración de 2.15× respecto a la ejecución secuencial.

Como ya se mencionó en la sección 4.3, la aceleración no es ideal, lo cual puede deberse a diversas razones que escapan al alcance de este PFG.

5.1.6. Robustez

Este objetivo se considera satisfecho. El sistema de validación de configuración sigue una política de *fail-fast* con agregación de errores. En lugar de abortar ante el primer problema detectado, o peor aún, iniciar una simulación costosa que fallará más adelante, el sistema recopila todas las violaciones de configuración y las notifica conjuntamente antes de iniciar cualquier cómputo.

5.2. Conclusiones

La energía solar es una de las fuentes renovables con mayor potencial de desarrollo en el futuro cercano. Producir herramientas, algoritmos y tecnología que permita, promueva y acelere su implementación es, ahora más que nunca, de suma importancia. Debido a la demanda energética actual, así como el estado general de los recursos y hábitats naturales, el uso y desarrollo de energías renovables es crucial.

SOLARIS demuestra ser, desde su fase inicial, una herramienta con gran potencial en la industria e investigación en el área de la energía solar. La posibilidad de generar datos sintéticos anotados de módulos fotovoltaicos no ideales, es decir, que son afectados por distintos fallos y modos de degradación, permite el entrenamiento y benchmarking de algoritmos de detección de estado de salud de paneles, cuyo desarrollo y refinamiento a su vez promueven la inversión y construcción de granjas solares, además de facilitar su longevidad y maximizar su producción.

Las técnicas de optimización y herramientas fundamentadas en principios físicos empleadas para el desarrollo del proyecto aseguran la correctitud de los datos generados y eficiencia del producto final.

Queda, de forma deliberada, en manos del usuario realizar una correcta implementación de los modelos de fallo y degradación que desea simular, para permitir que expertos en el área desarrollen modelos realmente precisos. Idealmente, SOLARIS puede llegar a ser un módulo central, que a través de aportaciones de múltiples expertos en el sector, permita la simulación realística de diversos efectos que pueden afectar a módulos fotovoltaicos.

5.3. Líneas futuras

El proyecto SOLARIS se encuentra en fases iniciales, habiendo demostrado la viabilidad y alcanzado los objetivos propuestos. En las siguientes fases del proyecto, existen varías líneas de mejora y extensión, algunas de las cuales se detallan a continuación:

- Transcribir a C++ o Rust partes del código para poder mejorar la estrategia de chunking a micro chunks dinámicos, solventando así el problema de fallos de caché y aumentando en gran medida la velocidad de simulación.
- Ejecutar sobre un entorno más consistente que el hardware personal, permitiendo obtener más información acerca del cuello de botella experimentado durante las ejecuciones paralelas.
- Incluir más efectos, si bien no perfectos, que sirvan como ejemplos de modelado y uso de la herramienta.
- Expandir el catálogo de configuraciones de módulos que se pueden simular añadiendo, por ejemplo, otros tipos de montura de paneles más allá de orientación fija, o agregación por hileras de los paneles.
- Introducir fallos en mppt y efectos similares que no modifican o afectan a los parámetros *CEC* pero sí a la potencia generada por el módulo.
- Investigar más en profundidad, consultar con expertos en el área y mejorar las reglas de agregación de efectos para una mayor fidelidad respecto a la realidad.
- Expandir el esquema de paralelización para soportar la ejecución en múltiples máquinas distribuidas permitiendo un mayor rendimiento.

Bibliografía

[1] U. Hijjawi, S. Lakshminarayana, T. Xu y M. Rahman, «A benchmarking study of instance segmentation methods for photovoltaic cell defect detection using electroluminescence images,» *Solar Energy*, vol. 303, pág. 114 083, 2026.

[2] G. Nassreddine, A. E. Arid, M. Nassereddine y O. A. Khatib, «Fault Detection and Classification for Photovoltaic Panel System Using Machine Learning Techniques,» *Applied AI Letters*, vol. 6, e115, 2025.

[3] A. Bakdi, A. Guichi, S. Mekhilef y W. Bounoua. «GPVS-Faults: Experimental Data for fault scenarios in grid-connected PV systems under MPPT and IPPT modes. »dirección: https://data.mendeley.com/datasets/n76t439f65/1 (accedido el: 26.06.2026), Mendeley Data, V1.

[4] B. Su, Z. Zhou y H. Chen, «PVEL-AD: A Large-Scale Open-World Dataset for Photovoltaic Cell Anomaly Detection,» *IEEE Transactions on Industrial Informatics*, 2022.

[5] I. E. Agency. «World Energy Outlook 2024. »dirección: https://www.iea.org/reports/world-energy-outlook-2024 (accedido el: 25.05.2026).

[6] I. E. Agency. «Electricity 2026 — Executive Summary. »dirección: https://www.iea.org/reports/electricity-2026/executive-summary (accedido el: 25.05.2026).

[7] U. E. I. Administration. «EIA projects nearly 50 % increase in world energy usage by 2050, led by growth in Asia. »dirección: https://www.eia.gov/todayinenergy/detail.php?id=41433 (accedido el: 25.05.2026).

[8] I. E. Agency. «World Energy Outlook 2025. »dirección: https://www.iea.org/reports/world-energy-outlook-2025 (accedido el: 25.05.2026).

[9] U. Nations. «Goal 7: Ensure access to affordable, reliable, sustainable and modern energy for all. »dirección: https://www.un.org/sustainabledevelopment/energy/ (accedido el: 25.05.2026).

[10] I. E. Agency. «Energy and AI — Executive Summary. »dirección: https://www.iea.org/reports/energy-and-ai/executive-summary (accedido el: 25.05.2026).

[11] Gartner. «Gartner Says Electricity Demand for Data Centers to Grow 16 % in 2025 and Double by 2030. »dirección: https://www.gartner.com/en/newsroom/press-releases/2025-11-17-gartner-says-electricity-demand-for-data-centers-to-grow-16-percent-in-2025-and-double-by-2030 (accedido el: 25.05.2026).

[12] N. M. Rusheen Patel y A. Patel, «The Environmental Impact of AI Servers and Sustainable Solutions,» *arXiv preprint arXiv:2601.06063*, 2025.

[13] R. for the Future, «Global Energy Outlook 2026: How the World Lost the Goal of 1.5°C,» *Resources for the Future Report*, 2026.

[14] Environmental y E. S. I. (EESI). «Data Center Power Demands Are Contributing to Higher Energy Bills. »dirección: https://www.eesi.org/articles/view/data-center-power-demands-are-contributing-to-higher-energy-bills (accedido el: 25.05.2026).

[15] P. C. Madara y H. Yousuf. «Building Integrated Photovoltaics: Technical and Aesthetic Prospects - Fig. 3, Components of a solar photovoltaic module. »dirección: https://www.researchgate.net/figure/Components-of-a-solar-photovoltaic-module-the-module-components-from-the-front-to-back_fig3_383431989 (accedido el: 18.06.2026).

[16] S. N. Laboratories. «PV Performance Modeling Collaborative (PVPMC). »dirección: https://pvpmc.sandia.gov/modeling-guide/2-dc-module-iv/single-diode-equivalent-circuit-models/ (accedido el: 20.06.2026).

[17] D. L. King, W. E. Boyson y J. A. Kratochvil, «Photovoltaic Array Performance Model,» Sandia National Laboratories, Albuquerque, NM, inf. téc. SAND2004-3535, 2004.

[18] S. K. W. De Soto y W. Beckman, «Improvement and validation of a model for photovoltaic array performance,» *Solar Energy*, vol. 80, n.º 1, págs. 78-88, 2006.

[19] C. E. Commission. «Solar Equipment Lists Program. »dirección: https://www.energy.ca.gov/programs-and-topics/programs/solar-equipment-lists (accedido el: 28.06.2026).

[20] M. Köntges et al., «Review of Failures of Photovoltaic Modules,» IEA Photovoltaic Power Systems Programme (IEA-PVPS), Task 13, inf. téc. IEA-PVPS T13-01:2014, 2014, ISBN 978-3-906042-16-9.

[21] M. Aghaei et al., «Review of degradation and failure phenomena in photovoltaic modules,» *Renewable and Sustainable Energy Reviews*, vol. 159, pág. 112 160, 2022.

[22] D. C. Jordan y S. R. Kurtz, «Photovoltaic Degradation Rates – An Analytical Review,» *Progress in Photovoltaics: Research and Applications*, vol. 21, n.º 1, págs. 12-29, 2013.

[23] M. Köntges et al., «Degradation and Failure Modes in New Photovoltaic Cell and Module Technologies,» IEA Photovoltaic Power Systems Programme (IEA-PVPS), Task 13, inf. téc. IEA-PVPS T13-30:2025, 2025, ISBN 978-3-907281-71-0.

[24] K. J. Wong. «Multithreading and Multiprocessing in 10 Minutes. »dirección: https://towardsdatascience.com/multithreading-and-multiprocessing-in-10-minutes-20d9b3c6a867/ (accedido el: 20.06.2026).

[25] K. S. Anderson, C. W. Hansen, W. F. Holmgren, A. R. Jensen, M. A. Mikofski y A. Driesse, «pvlib python: 2023 project update,» *Journal of Open Source Software*, vol. 8, n.º 92, pág. 5994, 2023.

[26] A. R. Jensen et al., «pvlib iotools—Open-source Python functions for seamless access to solar irradiance data,» *Solar Energy*, vol. 266, pág. 112 092, 2023, ISSN: 0038-092X. DOI: https://doi.org/10.1016/j.solener.2023.112092 dirección: https://www.sciencedirect.com/science/article/pii/S0038092X23007260

[27] E. C. Joint Research Centre. «Photovoltaic Geographical Information System (PVGIS). »dirección: https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis_en (accessed: 21.06.2026).

[28] J. R. C. European Commission et al. «Photovoltaics Geographical Information System: Status Report 2024. »dirección: https://data.europa.eu/doi/10.2760/2979595 (accessed: 21.06.2026), JRC141263.

[29] Open-Meteo. «Features - Open-Meteo.com. »dirección: https://open-meteo.com/en/features (accessed: 21.06.2026).

[30] Open-Meteo. «Historical Weather API - Open-Meteo.com. »dirección: https://open-meteo.com/en/docs/historical-weather-api (accessed: 21.06.2026).

[31] C. R. Harris et al., «Array programming with NumPy,» *Nature*, vol. 585, n.º 7825, págs. 357-362, 2020.

[32] W. McKinney, «Data Structures for Statistical Computing in Python,» en *Proceedings of the 9th Python in Science Conference*, S. van der Walt y J. Millman, eds., 2010, págs. 56-61. DOI: 10.25080/Majora-92bf1922-00a

[33] T. pandas development team, *pandas-dev/pandas: Pandas*, ver. v3.0.2, mar. de 2026. DOI: 10.5281/zenodo.19340003 dirección: https://doi.org/10.5281/zenodo.19340003

[34] A. S. Foundation. «Overview - Apache Parquet. »dirección: https://parquet.apache.org/docs/overview/ (accessed: 21.06.2026).

[35] A. S. Foundation. «Python – Apache Arrow v24.0.0. »dirección: https://arrow.apache.org/docs/python/ (accessed: 21.06.2026).

[36] T. Preston-Werner. «TOML: Tom's Obvious Minimal Language v1.0.0. »dirección: https://toml.io/en/v1.0.0 (accedido el: 24.06.2026).

[37] P. S. Foundation. «argparse — Parser for command-line options, arguments and subcommands. »dirección: https://docs.python.org/3/library/argparse.html (accedido el: 29.06.2026).

[38] K. Reitz y contributors. «Requests: HTTP for Humans – Requests 2.34.2 documentation. »dirección: https://requests.readthedocs.io/ (accedido el: 24.06.2026).

[39] C. O. da Costa-Luis, «tqdm: A Fast, Extensible Progress Meter for Python and CLI,» *Journal of Open Source Software*, vol. 4, n.º 37, pág. 1277, 2019.



Universidad Politécnica de Madrid

ETSI **SISTEMAS INFORMÁTICOS**